Mentor Caring System

Product Promotion + Software Engineering

Chen Letian 2330026013
Liu Bochen 2330026096
Yu Yicong 2330026201
Zeng Qikai 2330013039
Yue Kaiqi 2330026204
Yang Jingtian 2330026187

B07

Part 1

Introduction

From Manual Mentoring Management to a Centralized Role-Based System

From Design to Product

2.1 SRS

Requirement scope, use cases, representative flows

2.2 Architecture Design

Layered/MVC design, system architecture, subsystems

2.3 Detailed Design

Class diagram, representative classes, traceability

1. Introduction

From Manual Mentoring Management to a Centralized Role-Based System

3.1 Purposes & Positioning

Purposes Design, product positioning & target users

3.2 Feature & Advantages

Main features, advantages, user-friendliness, & code-related work

3.3 Live Demo

Mentor-centered workflow; show code runs as stated

Part 2.1

SE Part 1: SRS

Requirements, use cases, representative flows, and classes

Member 2 · YANG Jingtian

SRS Logic

Context Model → Use Case Diagram → State Transition (3.6 Communicate) → Build Classes → Class + Sequence Diagram.

1 Context Model	2 Use Case Diagram	3 State Transition Communicate	4 Classes identification	5 Class + Sequence Diagram
System boundary external dependencies	Actors + goals role permissions	Page states events + warnings	Nouns → objects attributes/operations	Static structure scenario interaction

Why 3.6 Communicate is the main state example

It contains both communication and appointment behavior: send/receive/reply messages, mentor time-slot setup, student slot selection, and warning paths.

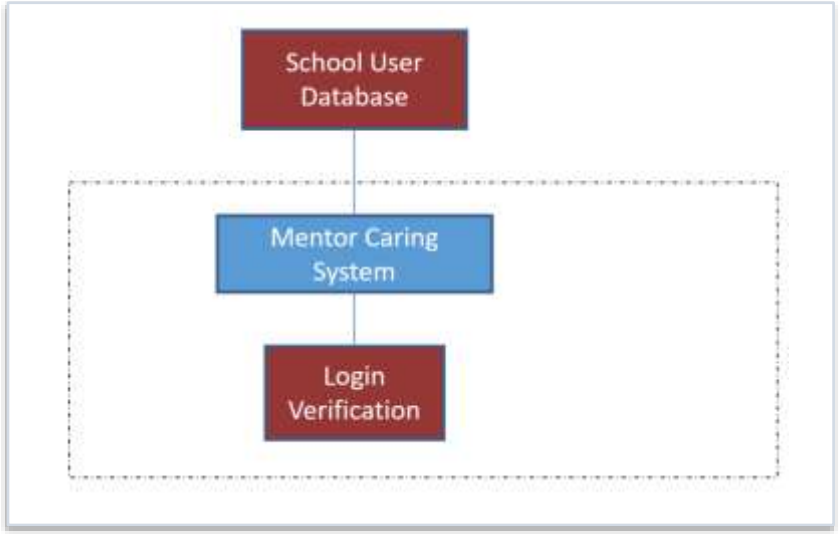
Why the class model is explained in detail

Class construction shows how SRS nouns become domain objects, how attributes and operations are selected, and how relationships support the whole MCS workflow.

Context Model + Use Case Diagram

These two models define the system boundary and the role-based goals before behavior modeling.

Context model: what is inside MCS?



Scope decision

MCS handles mentoring activities. Login verification and school user data are external dependencies, not the main product logic.

Use case overview: who uses which service?

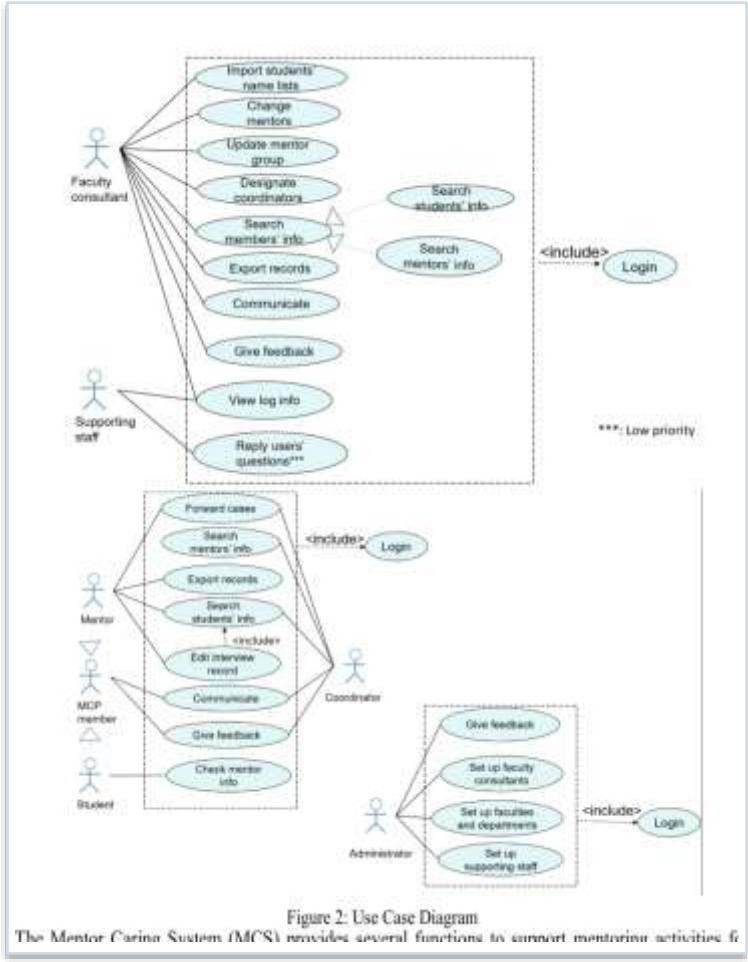


Figure 2: Use Case Diagram

The Mentor Caring System (MCS) provides several functions to support mentoring activities fo

Actors

Faculty Consultant, Mentor, Student, Coordinator, Administrator, Supporting Staff.

Representative use case

Communicate is chosen because it covers both message exchange and appointment arrangement.

Transition

Use cases say what users want; state transition diagrams show how the UI/system behaves.

State Transition Diagram: Communicate

This part shows behavior states, events, guards and exception paths for message and appointment communication.

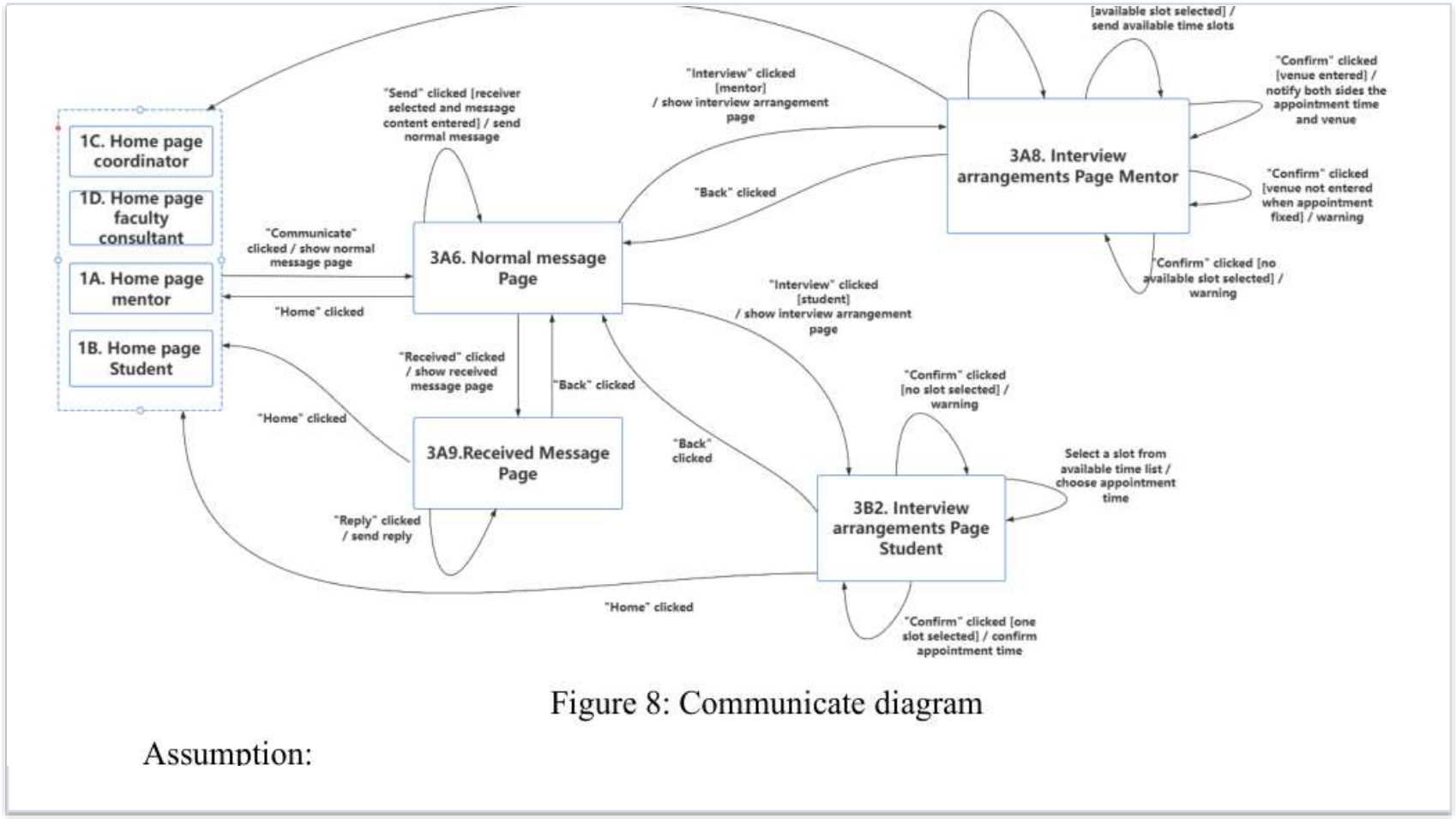


Figure 8: Communicate diagram

Assumption:

Normal message path

Home -> Normal Message -> Send / Received / Reply.

Mentor appointment path

Mentor enters available slots and venue, then confirms.

Student appointment path

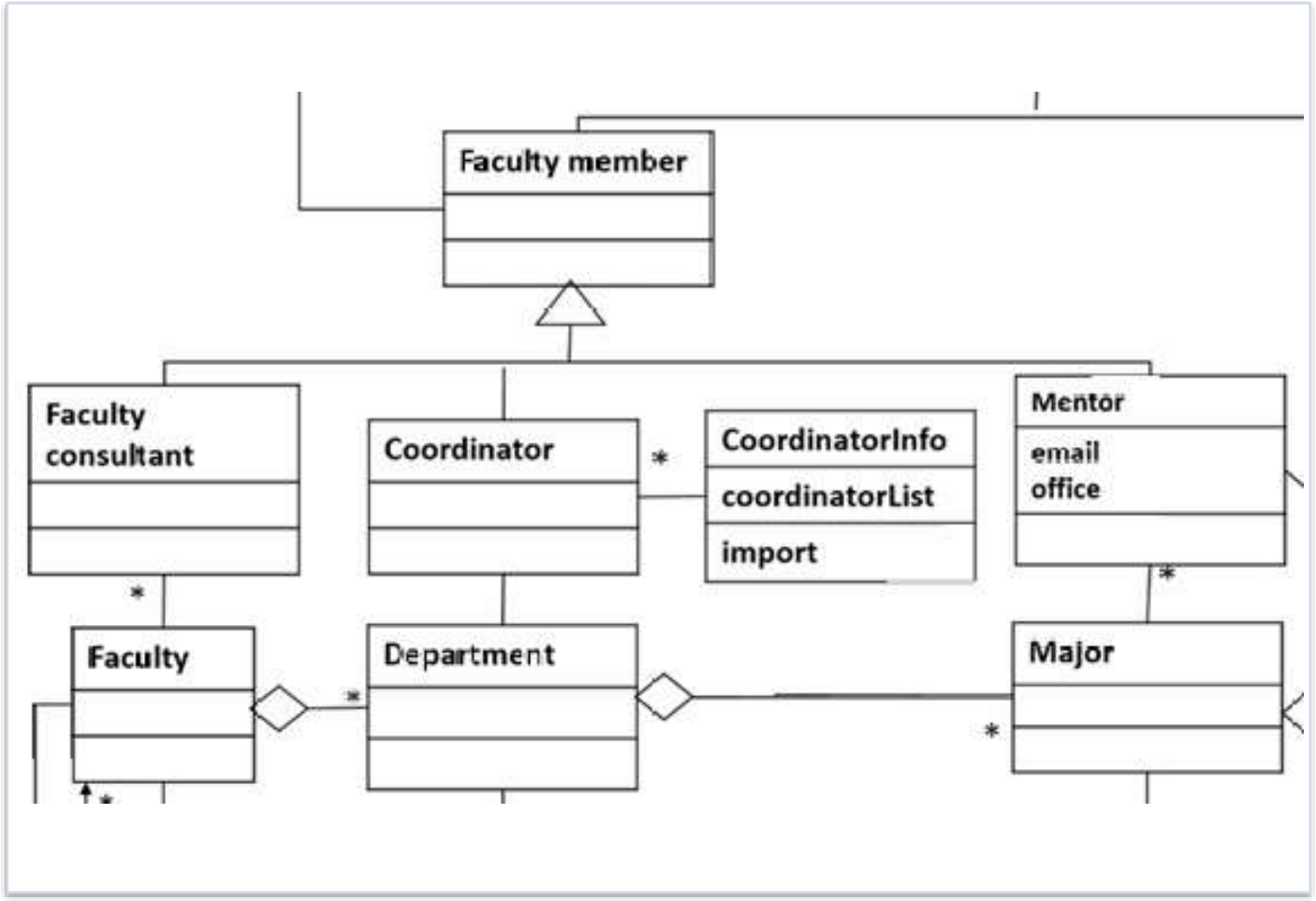
Student selects one available slot and confirms appointment time.

Alternative paths

Warnings are produced when receiver/content, slot or venue is missing.

Building Classes from Requirements

Extract nouns from the description and construct classes (attributes, operations) and relationships based on the use case diagram.



1. Identify superclass

Repeated staff roles are abstracted as Faculty member.

2. Specialize roles

Faculty consultant, Coordinator and Mentor inherit from Faculty member because they share staff identity but own different duties.

3. Add support classes

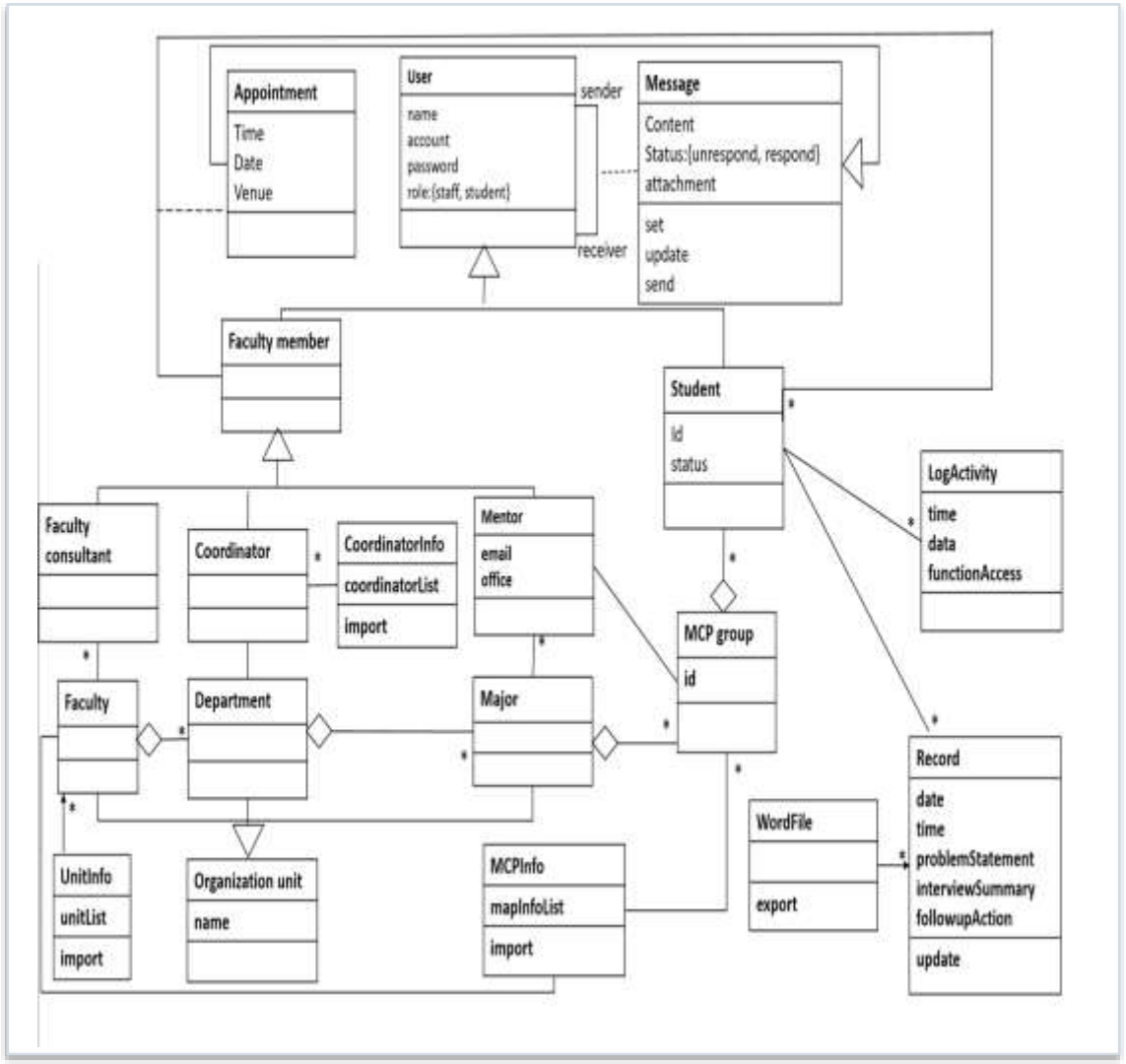
CoordinatorInfo stores coordinatorList and import(), supporting the designation/import requirement.

4. Connect organization units

Faculty, Department and Major form the university structure; their associations explain allocation and group management.

Class + Sequence Diagram

The final overall class and sequence diagram is as follows.



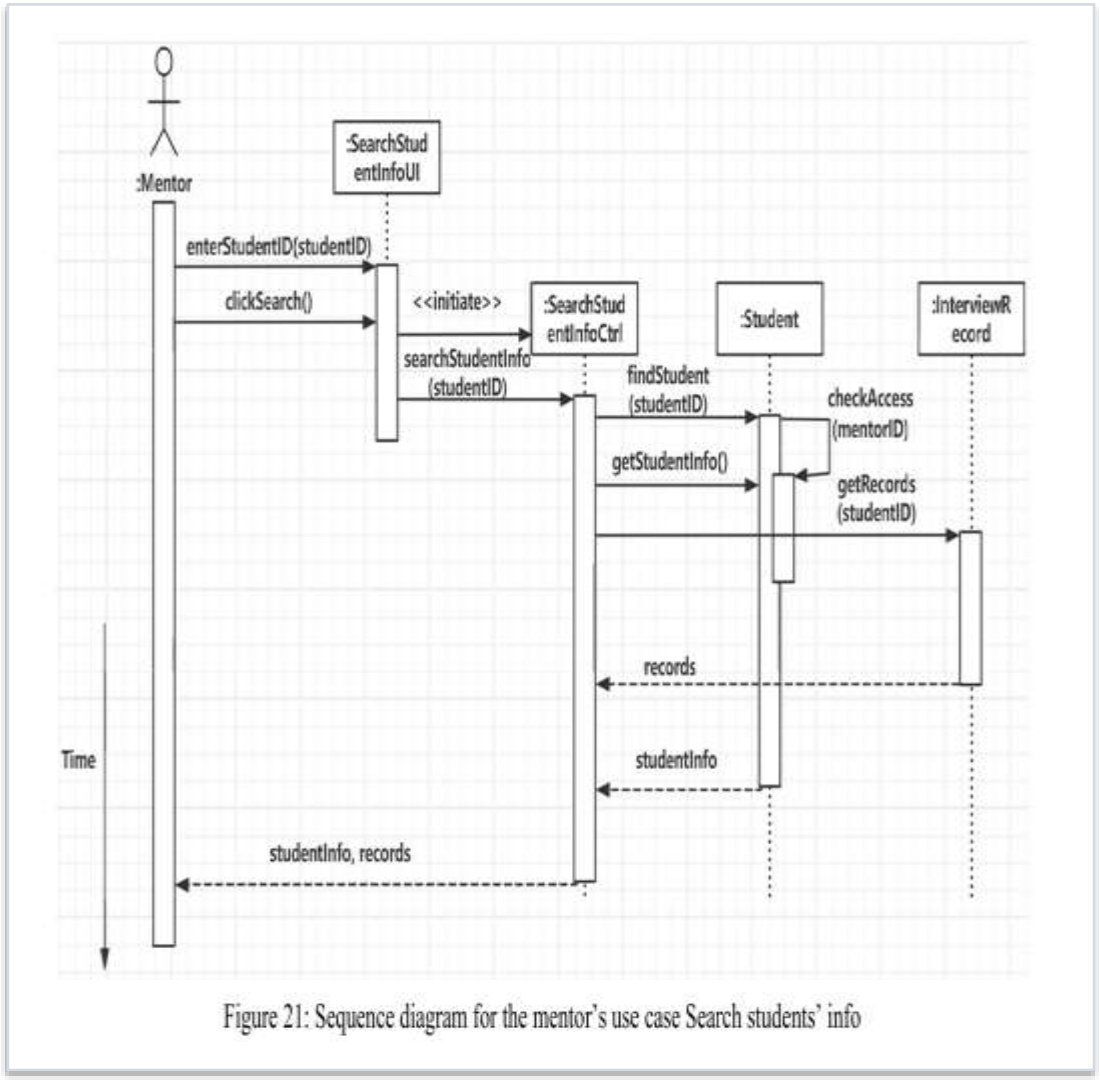


Figure 21: Sequence diagram for the mentor's use case Search students' info

Part 2.2

SE Part 2: Architecture Design

Architecture model, subsystem decomposition, and database support for MCS

Member 4 · Liu Bochen

Architecture Design Logic

Show how requirements become a maintainable system structure.

Design model

Layered architecture + MVC

System architecture

Four major subsystems

Database support

Tables support core data and associations

Subsystem design

Boundary / control / entity / mapper responsibilities

Core claim

Divide into

- Organization & Member
- Communication & Appointment
- Record & Reporting
- Audit & Support subsystems

Keep UI, workflow logic, and data operations independent.

Design Model: Layered Architecture + MVC

Explain the architecture model.

Interface

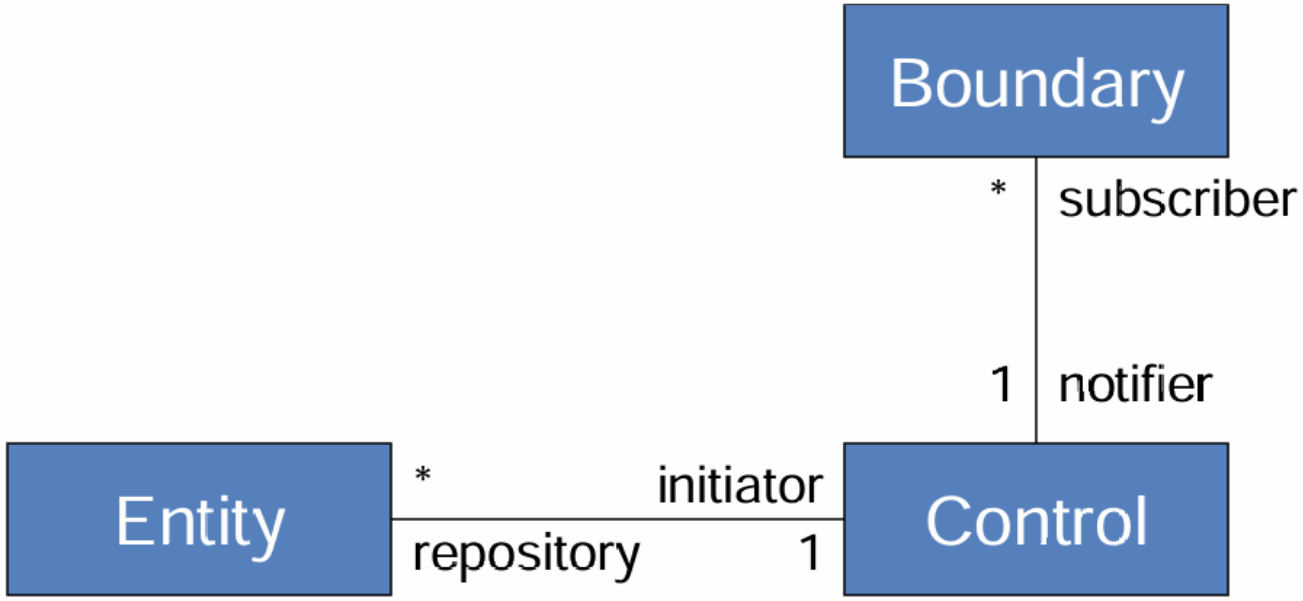
User interfaces and Flask boundary blueprints

Application Logic

Control classes handle workflows and rules

Storage

Mappers and ORM entities persist system data



MVC in MCS

Boundary works as the View

Control works as the Controller

Entity works as the data access part.

This separation keeps each parts independently

System Architecture Diagram

Overall architecture diagram explain.

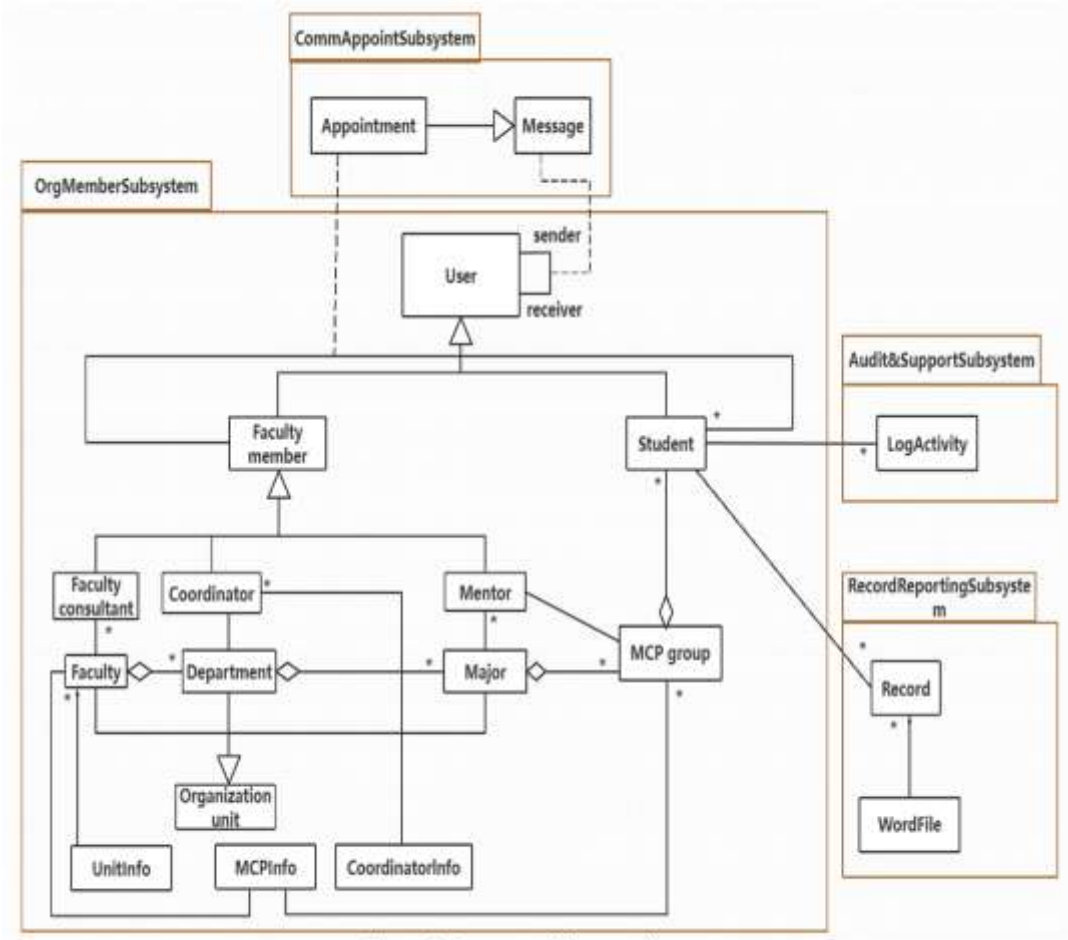


Figure 3: System architecture diagram

How to explain

1.Member, role, and group data are the core.

2.Four subsystems map to the main feature groups.

3.Communication and records reuse shared member data.

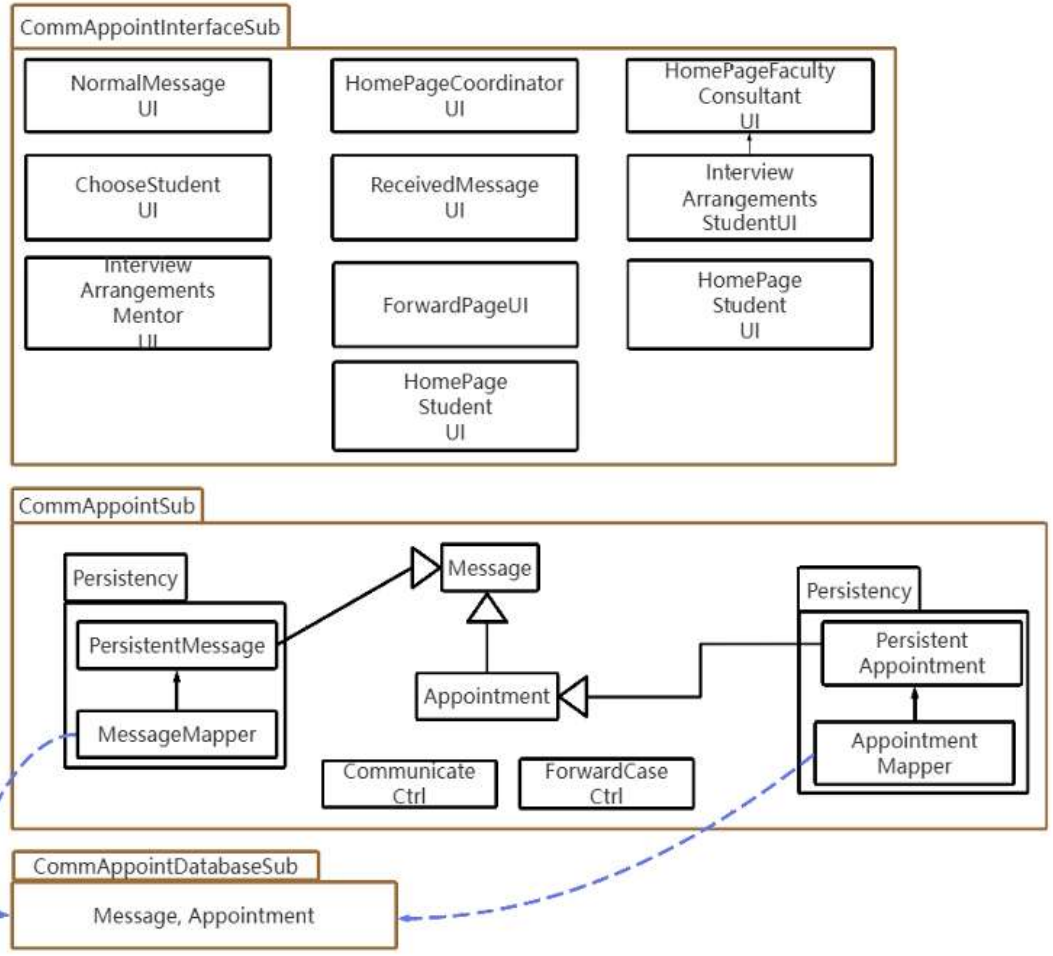
4.Audit/support improves traceability.

Why it matters

High cohesion
Low coupling

Subsystem Design Overview

Subsystem are divided into four subsystems with four main functions



Role-based Interface

Different UI pages are provided for different users,.

Core Business Objects

Four major subsystems

Controller

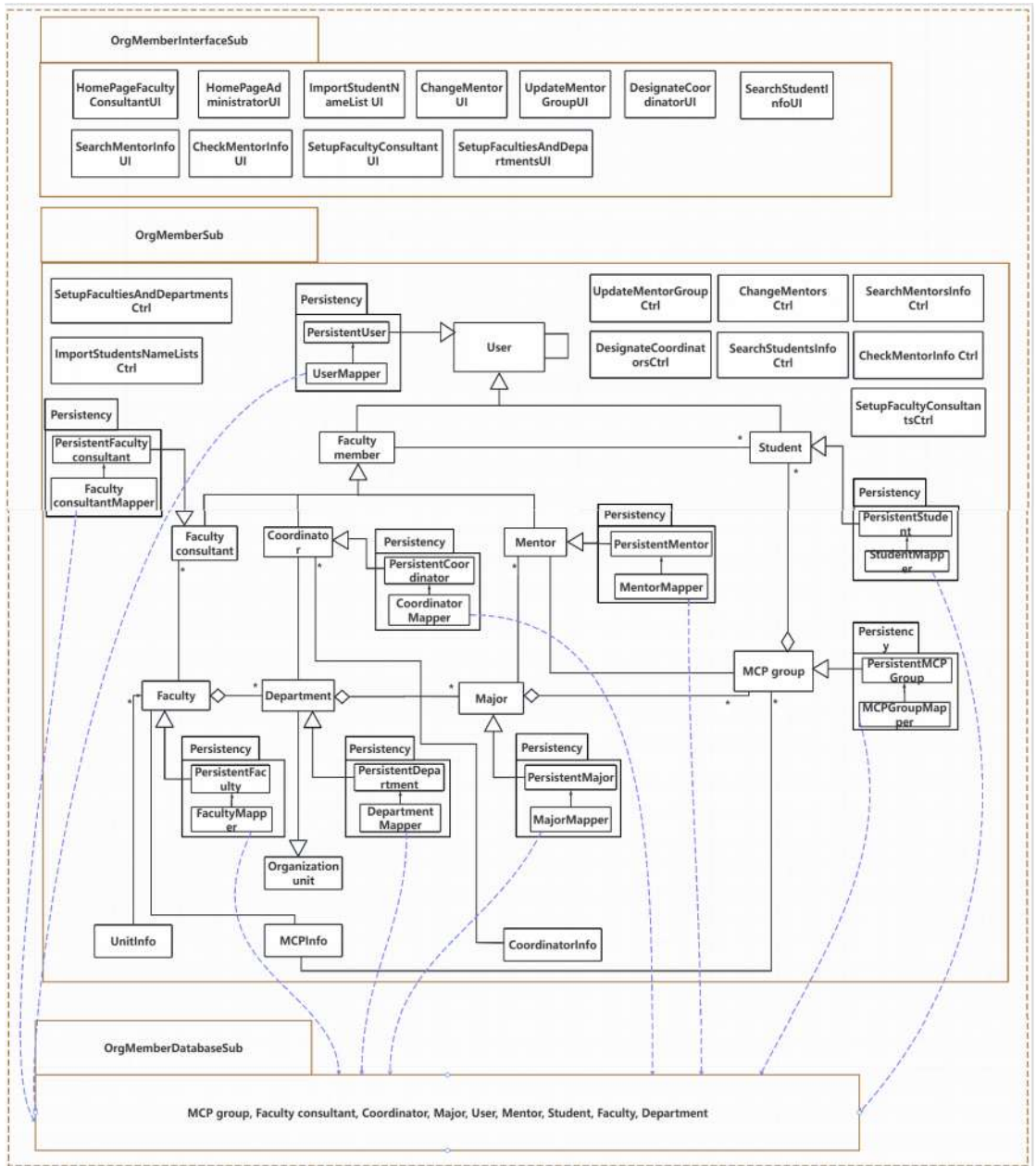
Tables support core data and associations

Persistence & Database

Store Message and Appointment data.

Subsystem Design Overview

Subsystem are divided into four subsystems with four main functions



Organization& Member

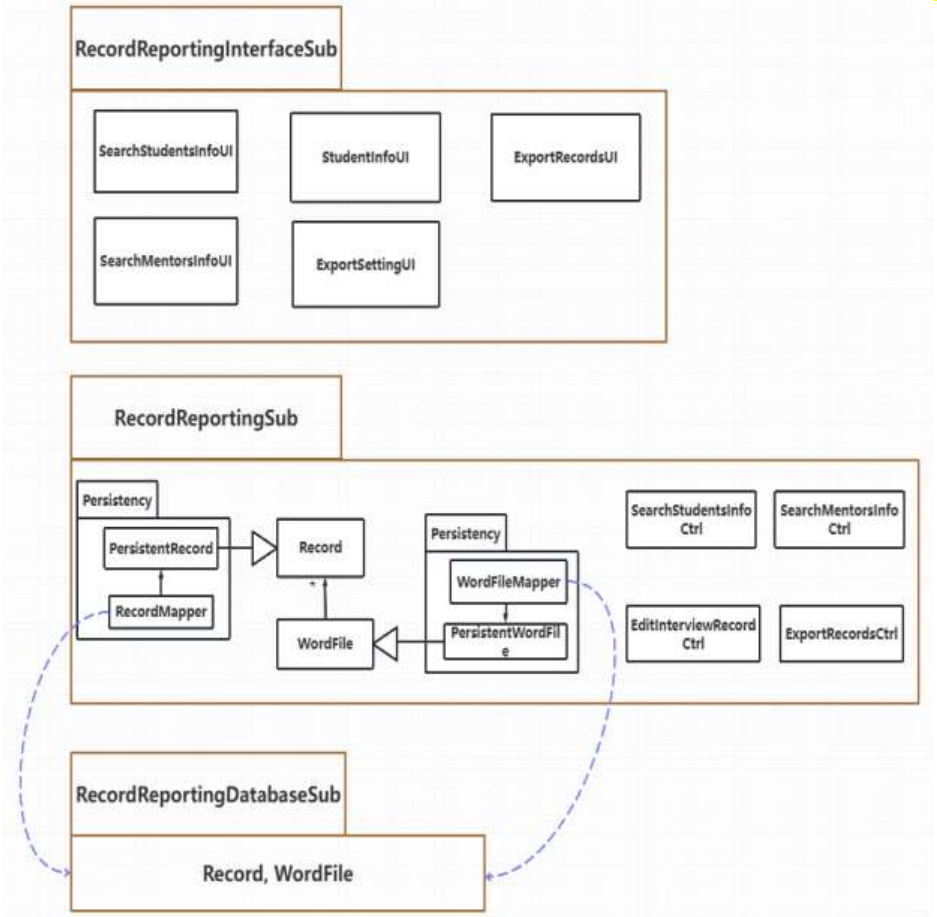
Manages students, mentors, coordinators.

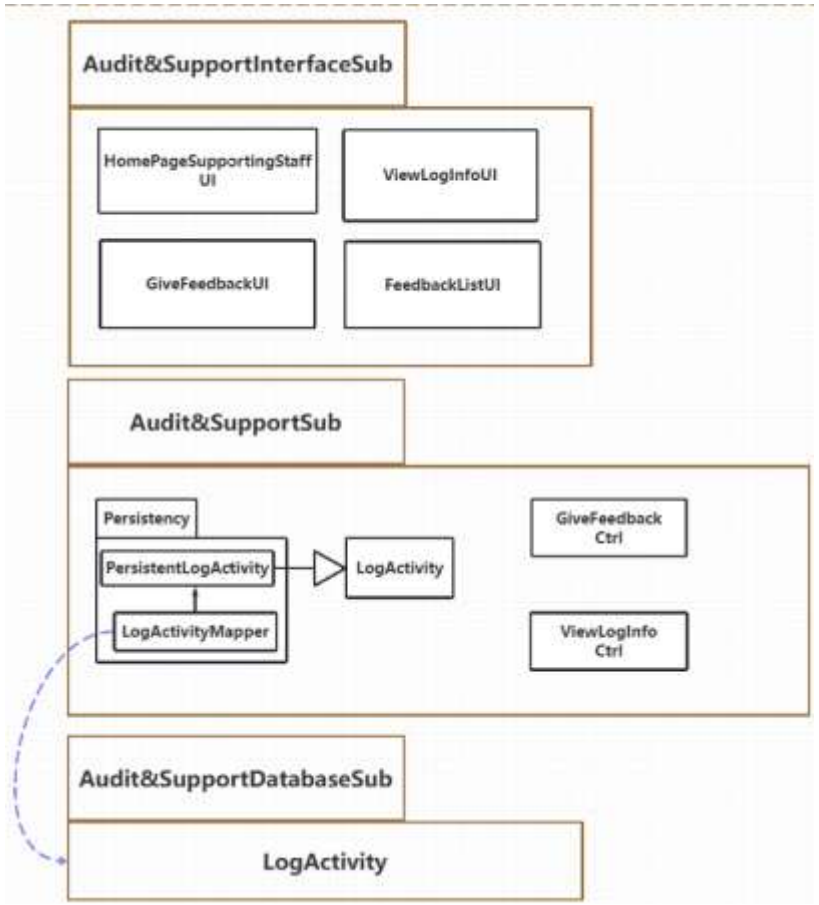
Defines department, major, and group relationships

Supports member import, role update, and group assignment

Subsystem Design Overview

Subsystem are divided into four subsystems with four main functions





Record & Reporting

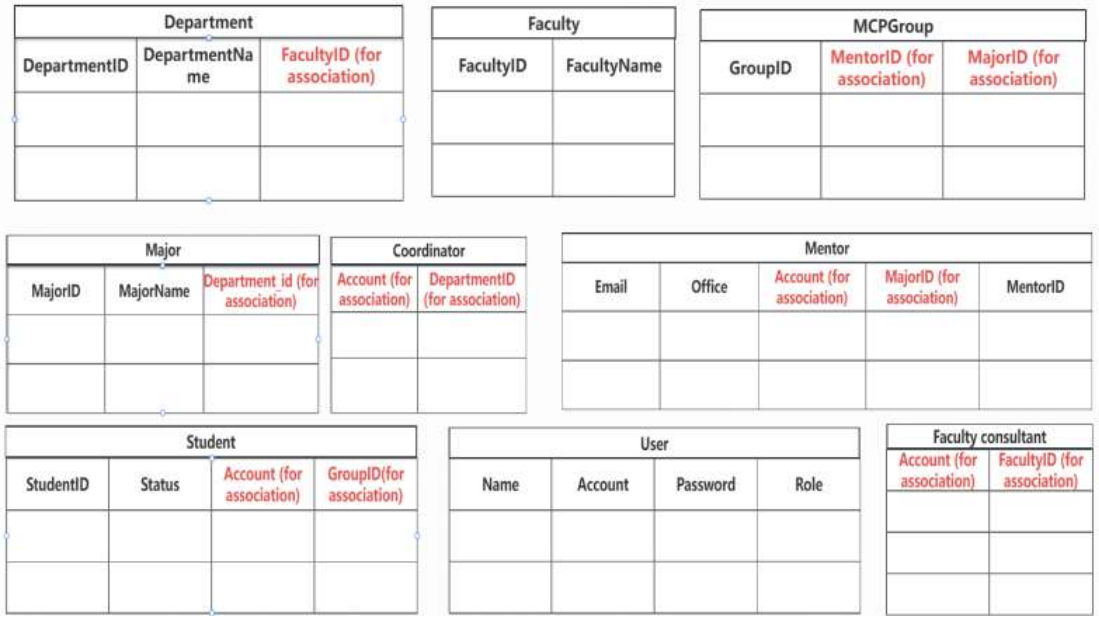
Interview records and export

Audit & Support

Logs and feedback/support

Database Support

Database support for architecture.



Department		
DepartmentID	DepartmentName	FacultyID (for association)

Faculty	
FacultyID	FacultyName

MCPGroup		
GroupID	MentorID (for association)	MajorID (for association)

Major		
MajorID	MajorName	Department id (for association)

Coordinator	
Account (for association)	DepartmentID (for association)

Mentor				
Email	Office	Account (for association)	MajorID (for association)	MentorID

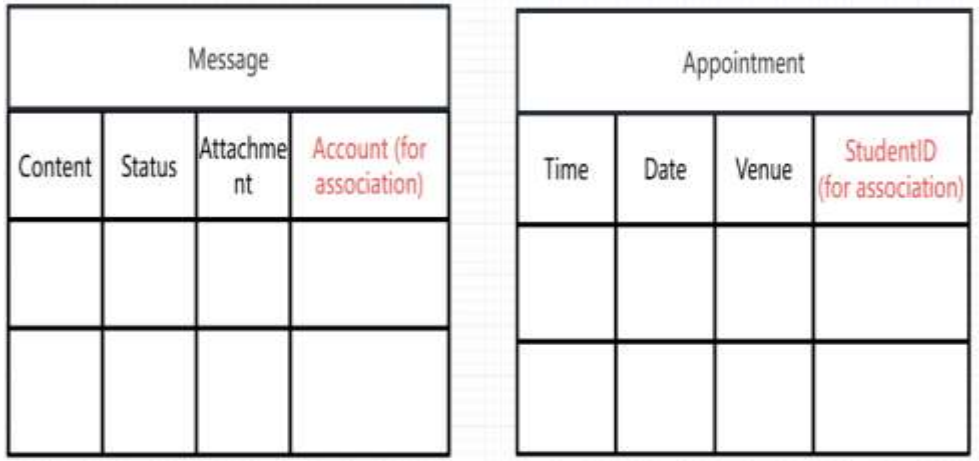
Student			
StudentID	Status	Account (for association)	GroupID(for association)

User			
Name	Account	Password	Role

Faculty consultant	
Account (for association)	FacultyID (for association)

Member&Organization DB

Core member and organization data for users, roles, mentors, students, and groups.



Message			
Content	Status	Attachment	Account (for association)

Appointment			
Time	Date	Venue	StudentID (for association)

Message & Appointment DB

Communication messages and appointment records

Database Support

Four Database support for architecture.

Record					
Date	Time	ProblemStatement	InterviewSummary	FollowupAction	StudentID(for association)

WordFile

Record & WordFile DB
Interview records and exported report files

LogActivity			
Time	Data	FunctionAccess	StudentID(for association)

Audit&Surpport DB
Operation logs and feedback data

Database tables provide persistent storage for core entities and cross-subsystem data associations.

Part 2.3

SE Part 3: Detailed Design + Testing

Class diagram, representative classes, design-to-code traceability and software testing

Member 4 · Zeng Qikai

Class Diagram Evolution to Detailed Design

The detailed design refines classes, attributes, operations and relationships.

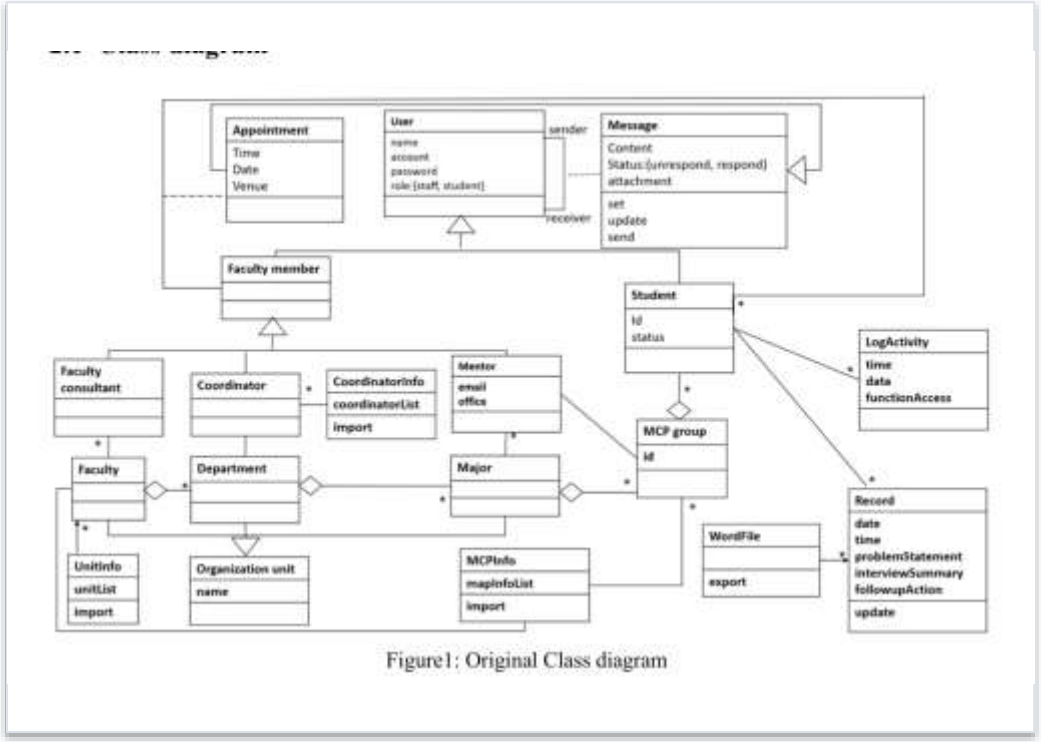
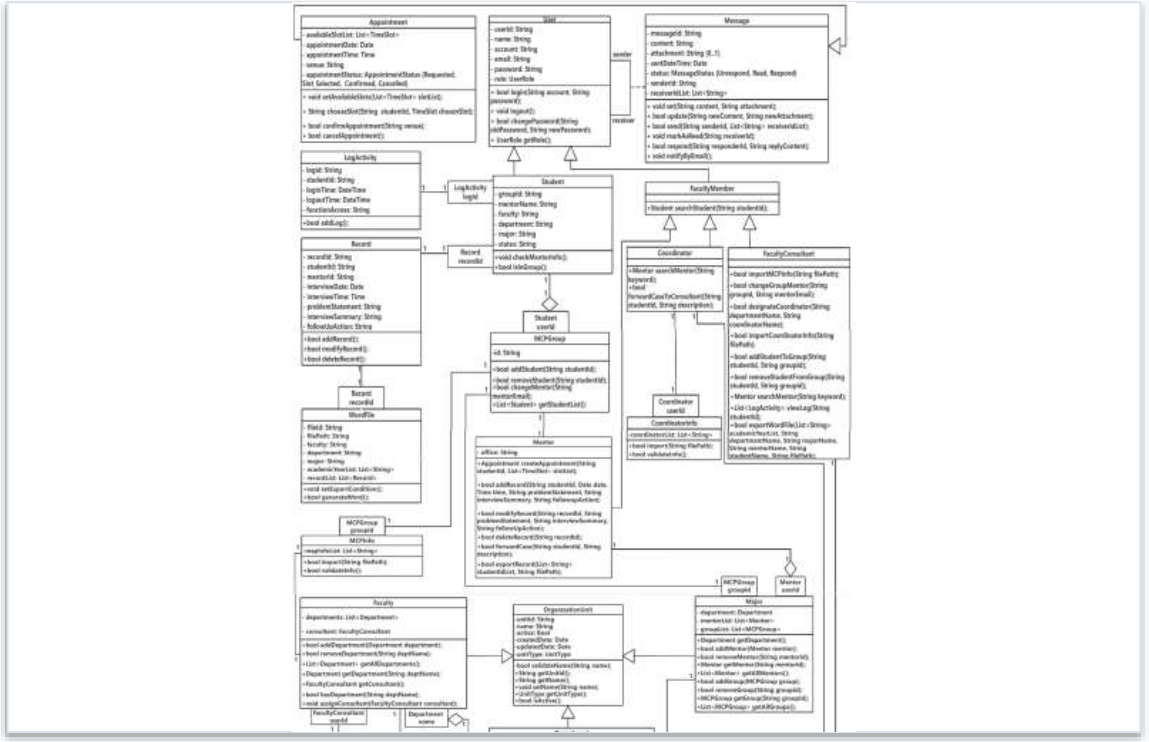


Figure1: Original Class diagram

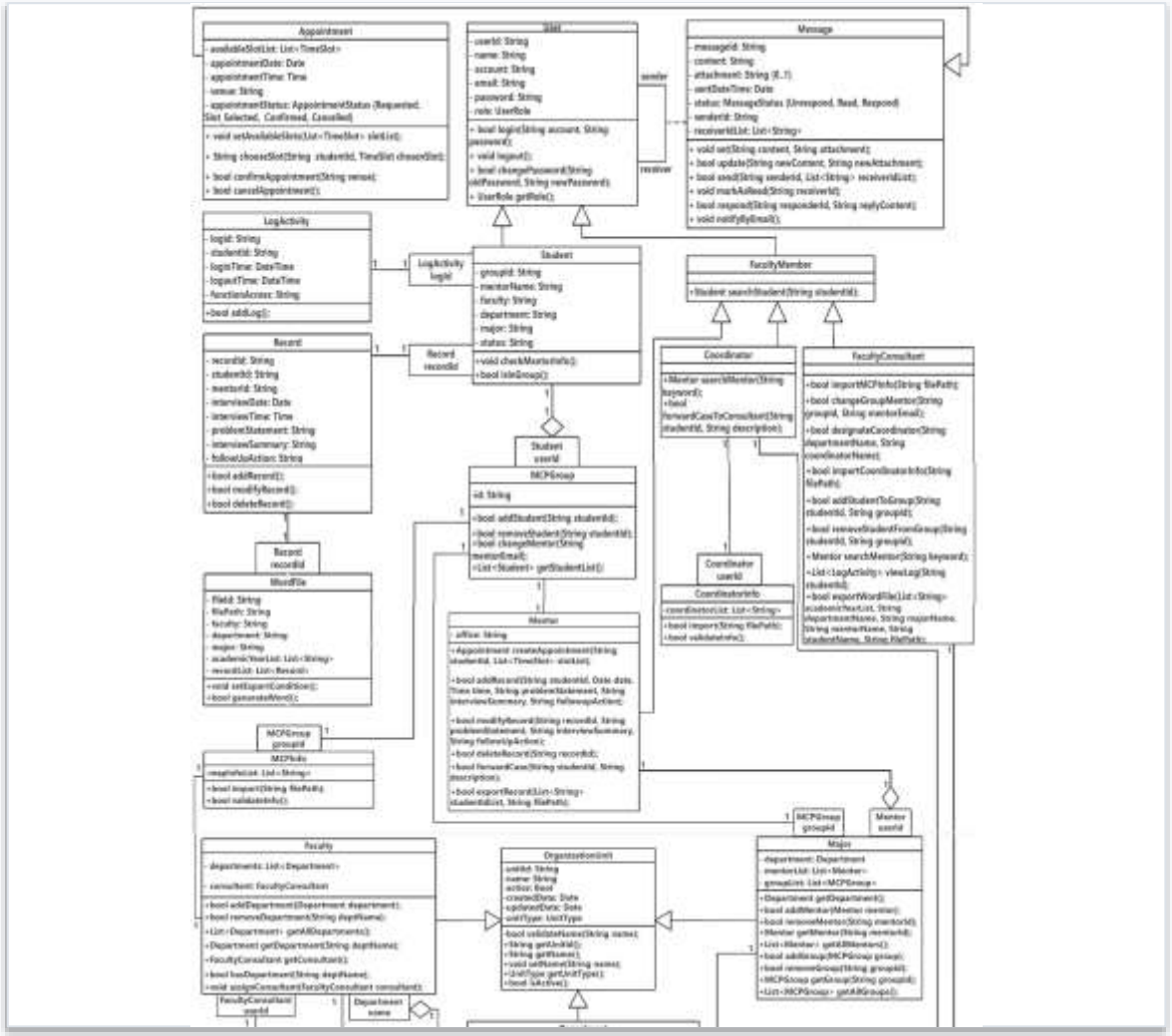


Analysis point

The updated class diagram refines the original model by clarifying user roles, adding detailed attributes and operations, and improving relationships among users, groups, messages, appointments, records, and exported files. This makes the design closer to implementation.

Updated Detailed Design Class Diagram

This diagram presents the final class-level structure of MCS after refinement



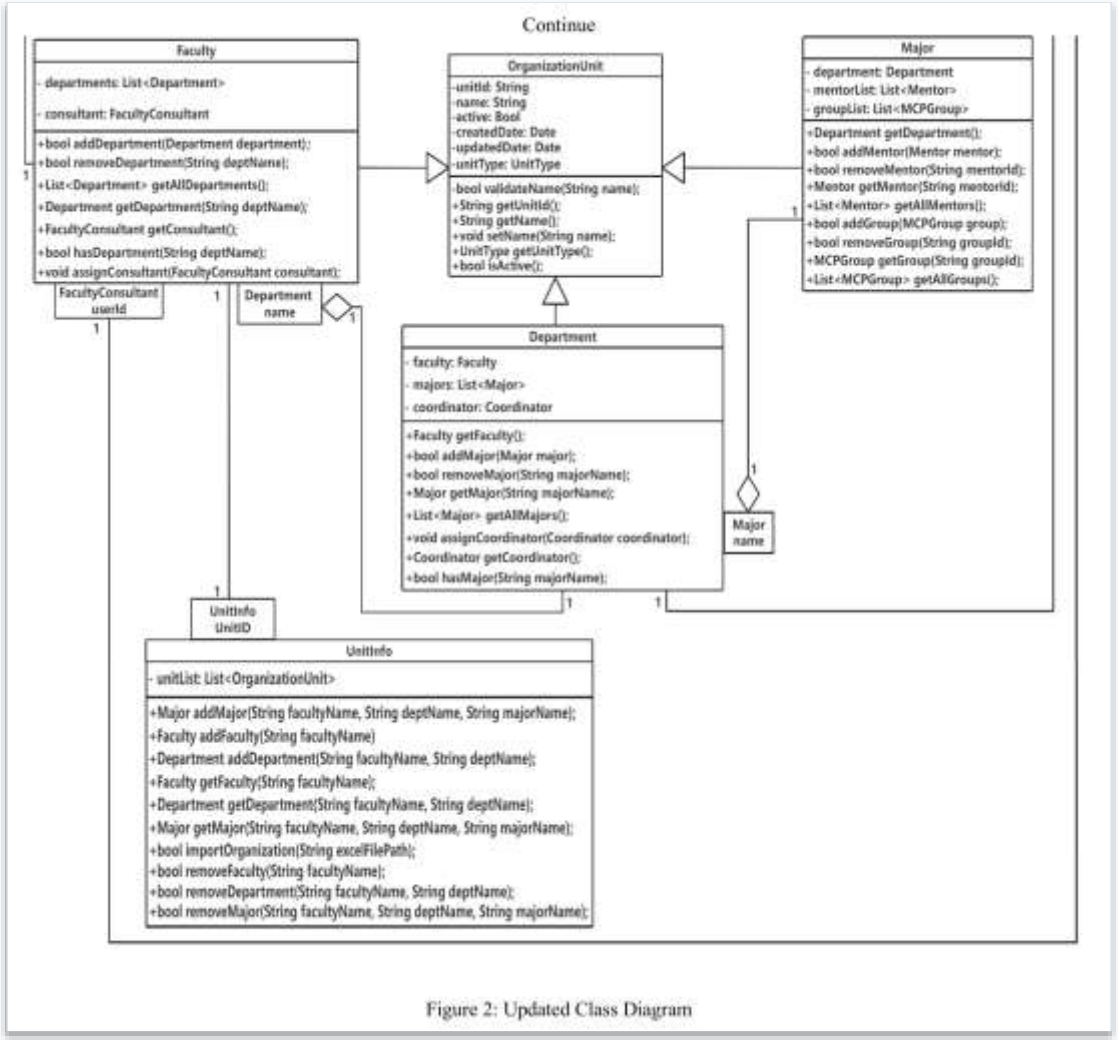
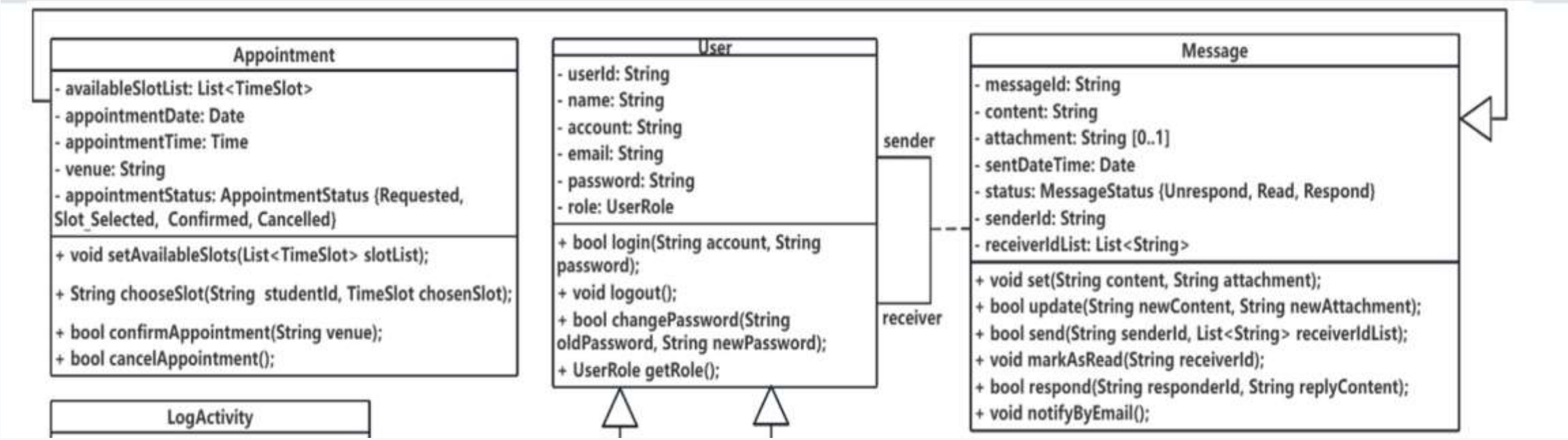


Figure 2: Updated Class Diagram

Representative Classes: Communication & Appointment

Use one representative part to explain detailed design clearly.



Appointment

Manages available interview slots

Supports slot selection and confirmation

Handles appointment status changes

Message

Stores message content and participants

Supports sending, replying, and reading

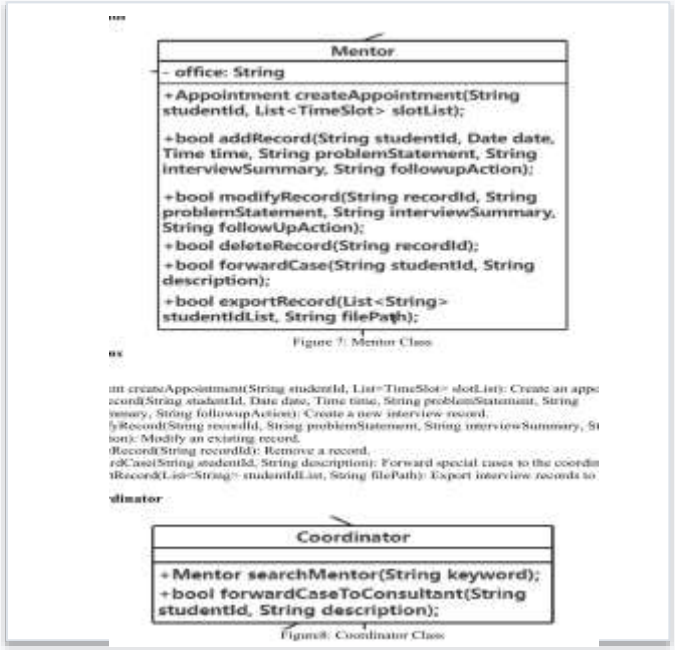
Enables mentor-student communication

Why this pair matters

Message and Appointment, together with User relationships, support the mentor-student workflow for communication and appointment arrangement.

Supporting Classes: Records, Roles and Export

These classes support interview record management, responsibility control, and report gengeration



Mentor / Coordinator

Mentor manages interview records. Coordinator handles forwarded cases and supports follow-up.

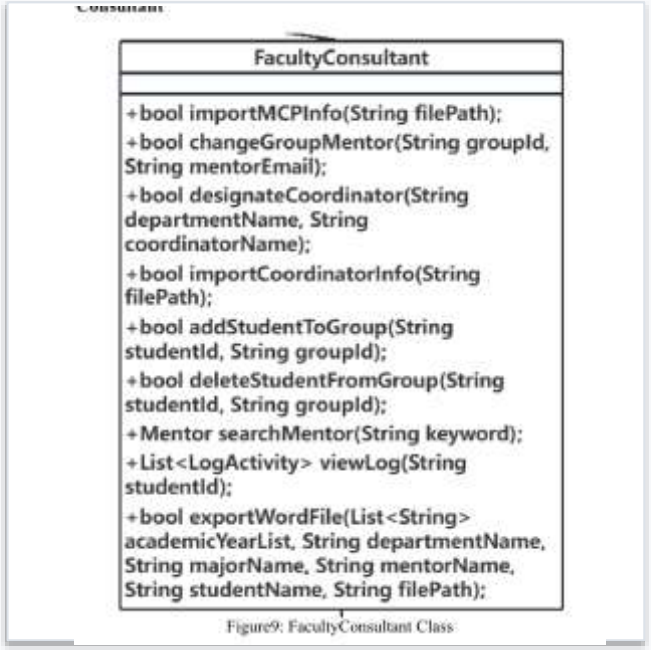


Figure9: FacultyConsultant Class

FacultyConsultant

Manages MCP groups, changes mentors, designates coordinators, and exports selected records.

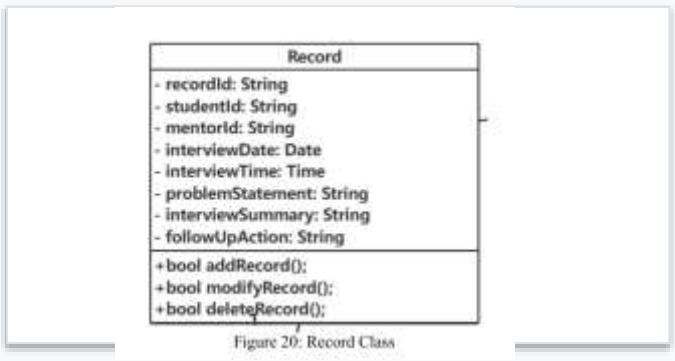


Figure 20: Record Class

Record

Stores interview information, including student, mentor, problem statement, interview summary, and follow-up action.

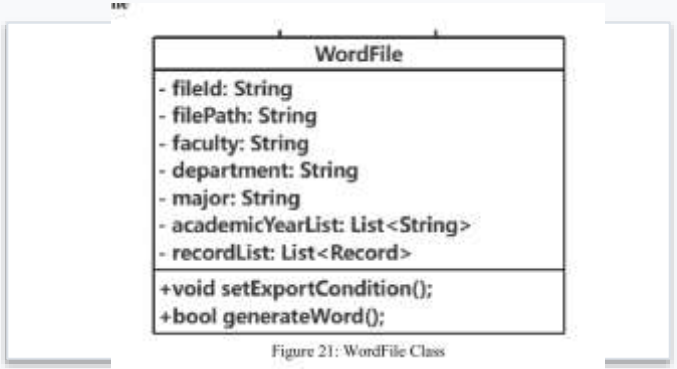


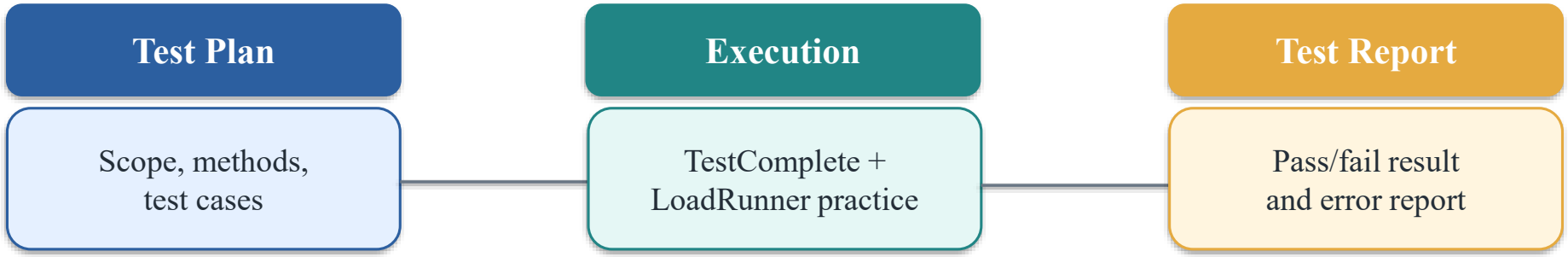
Figure 21: WordFile Class

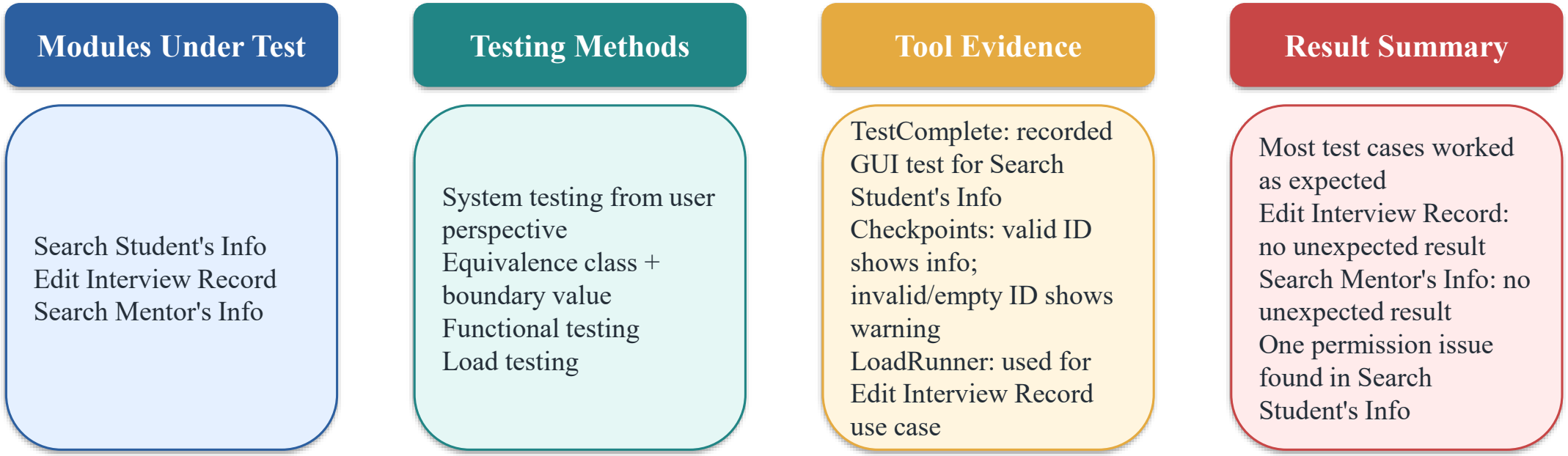
WordFile

Uses selected conditions and records to generate the final Word report.

Testing Verification

We tested three core use cases with black-box tests and tools.





Test Result

A concrete result selected from the Search Student Info testing evidence.

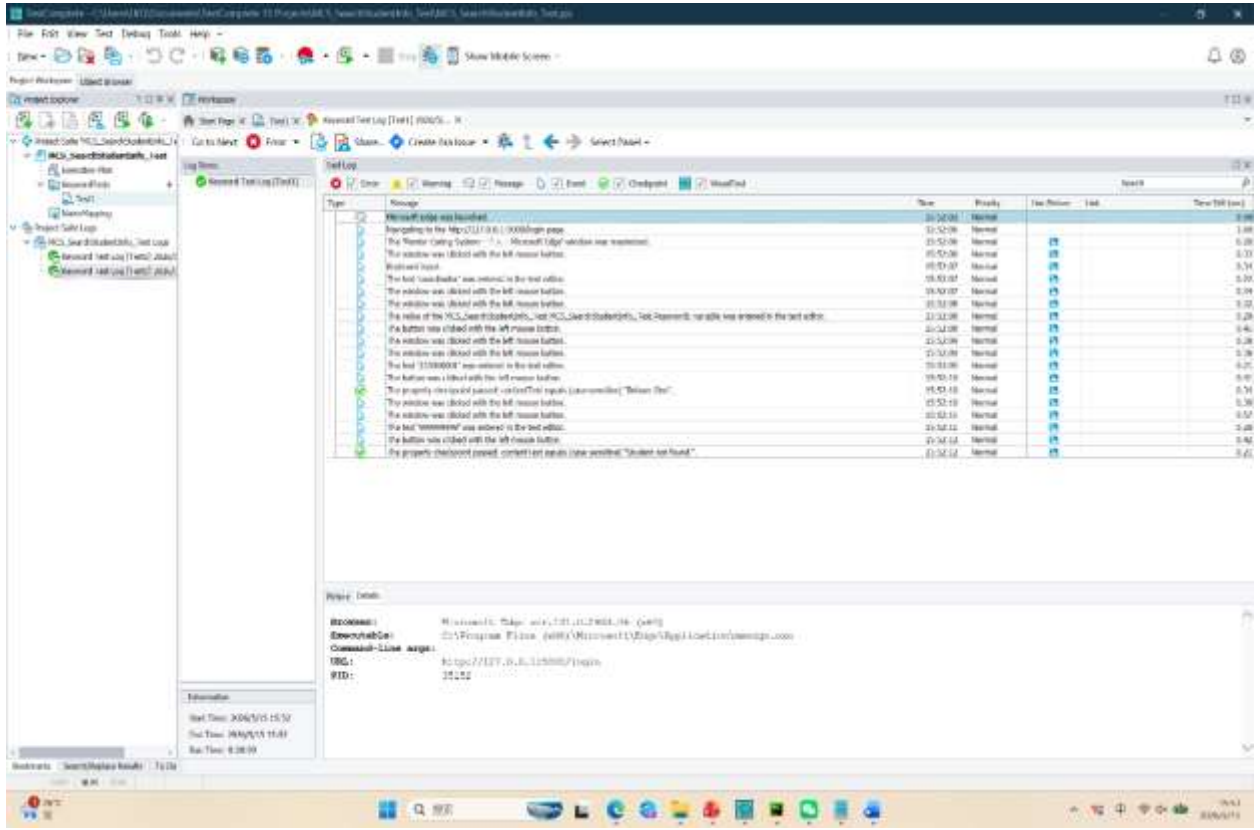
Selected Result: SSI-EC-06

Test case
Mentor searches for a student outside his/her permission scope.

Expected Result
A mentor should not access a student outside his/her permission scope.

Actual Result
The mentor could enter the Student Detail Page, but was blocked only when opening interview records.

Tool Evidence: TestComplete Test Log



Traceability: Requirements -> Design -> Code

This slide links SE documents with the implementation.



Example trace

Requirement → subsystem → User / Message / Appointment classes → message & appointment code modules → live demo workflow

Part 3.1

ASDW: Purposes & Positioning

Purposes Design, product positioning & target users

Member 1 · Chen Letian

Purposes Design

Why is a dedicated system needed?

Before MCS

Scattered information

Manual coordination

Inconsistent records

Repeated report preparation

After MCS

Centralized platform

Role-based workflow

Traceable records

Faster report export

One platform for mentoring groups, records, communication, appointments, and reports.

Mentor Caring System

One platform for the complete mentoring workflow

CENTRALIZED

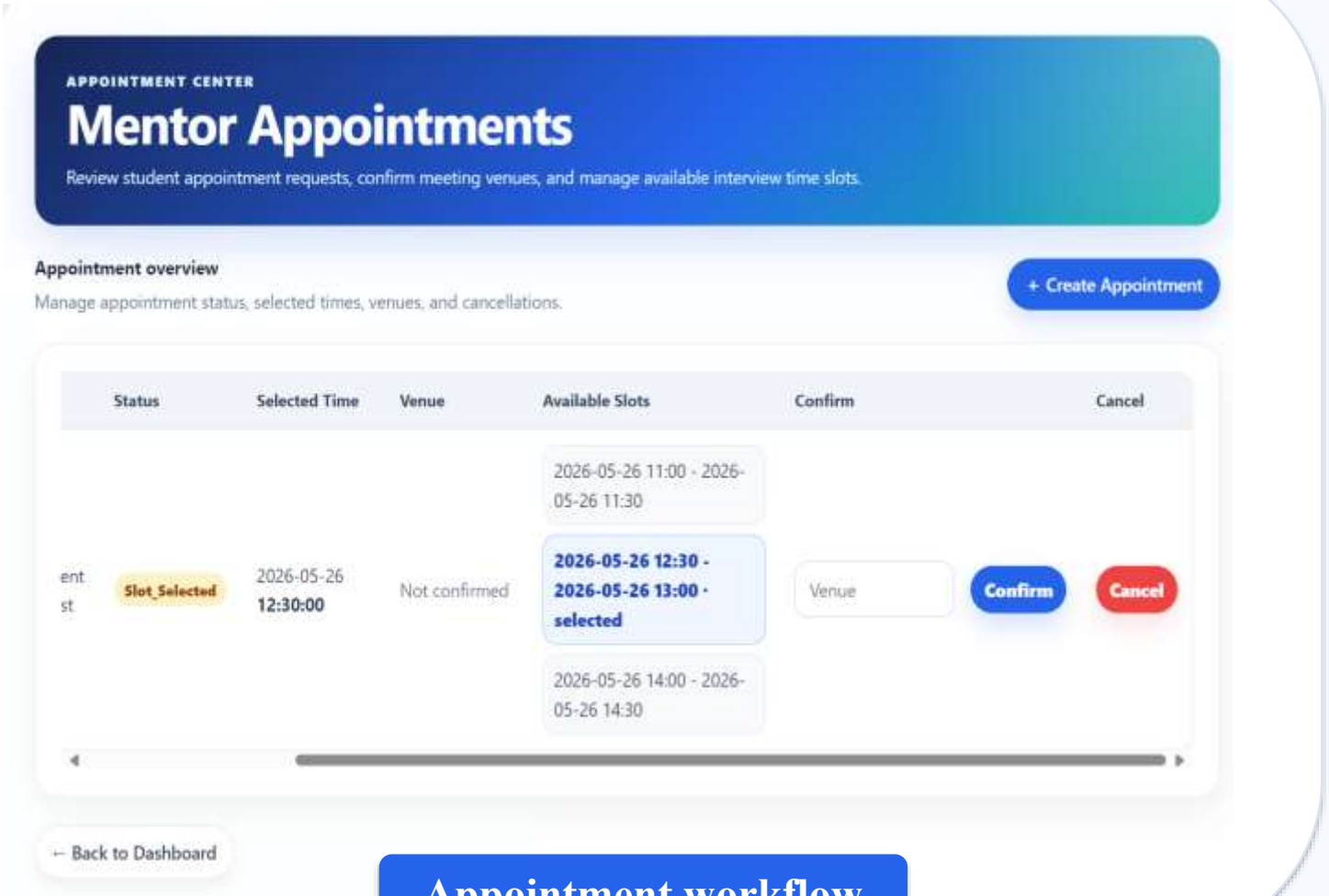
ROLE-BASED

TRACEABLE

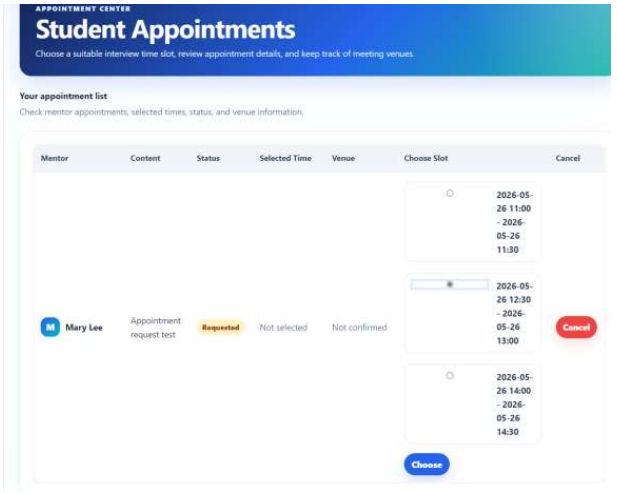
MCS

Showcase

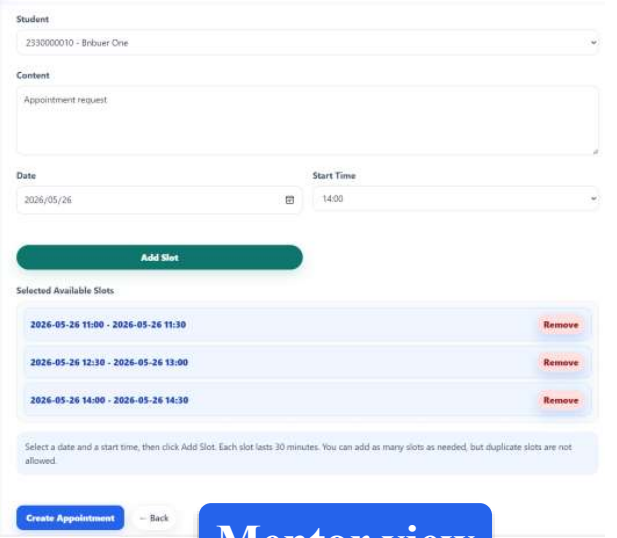
A centralized, role-based, and user-friendly web system for BNBU Mentor Caring Programme.



Appointment workflow



Student view



Mentor view

Product Positioning & Target Users

MCS is a centralized role-based platform for managing mentoring activities.

Product positioning

- Web-based centralized platform
- Connect group management, mentor allocation student records, communication, appointment arrangement, & report export
- Help different roles work on the same mentoring workflow

Main speaking focus: Faculty Consultant, Mentor, & Student

These three roles best represent the main mentoring workflow.

Faculty Consultant

Import student lists
Manage mentor groups
Change mentors
Export records

Mentor

Search students
Edit interview records
Communicate with students
Arrange appointments

Student

Check mentor information
Receive & Reply messages
Choose appointment slots

Coordinator

View student / mentor information
Communicate with users
Handle forwarded cases

Administrator

Set up faculties, departments, consultants & supporting staff

Supporting Staff

View logs
Reply to feedback
Support system operation

Part 3.2

ASDW: Feature & Advantages

Main features, advantages, user-friendliness, and code-related work

Member 5 · Yue Kaiqi

Product Advantages

1

Centralized

No more scattered files and messages.

2

Role-based

Different users get focused tasks.

3

Easy actions

Buttons, tables, and forms make next steps obvious.

4

Traceable

Logs and status support supervision.

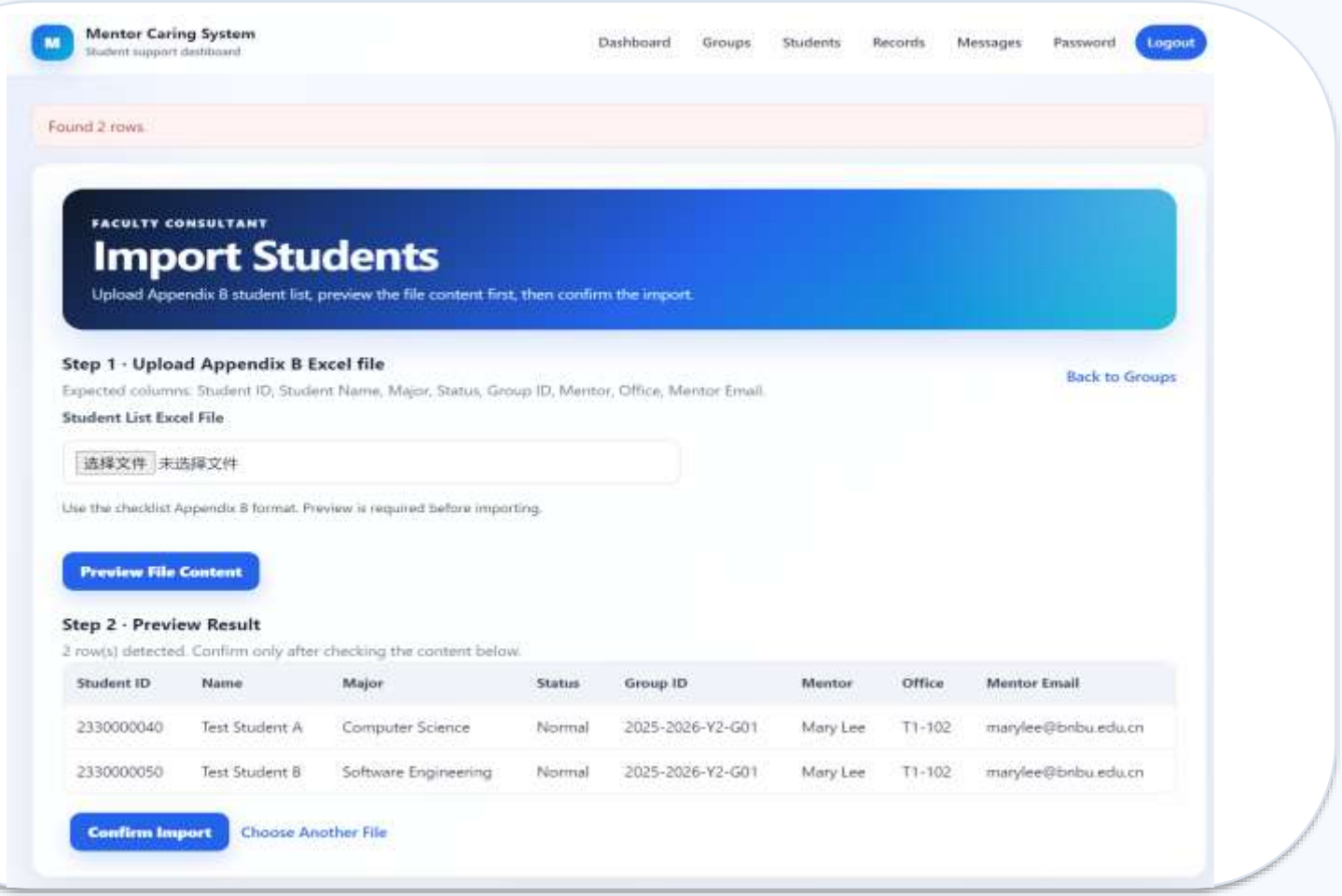
5

Report-ready

Export selected records to Word output.

Feature 1 · Centralized Data

Import students once and build a reliable student database



Student Import

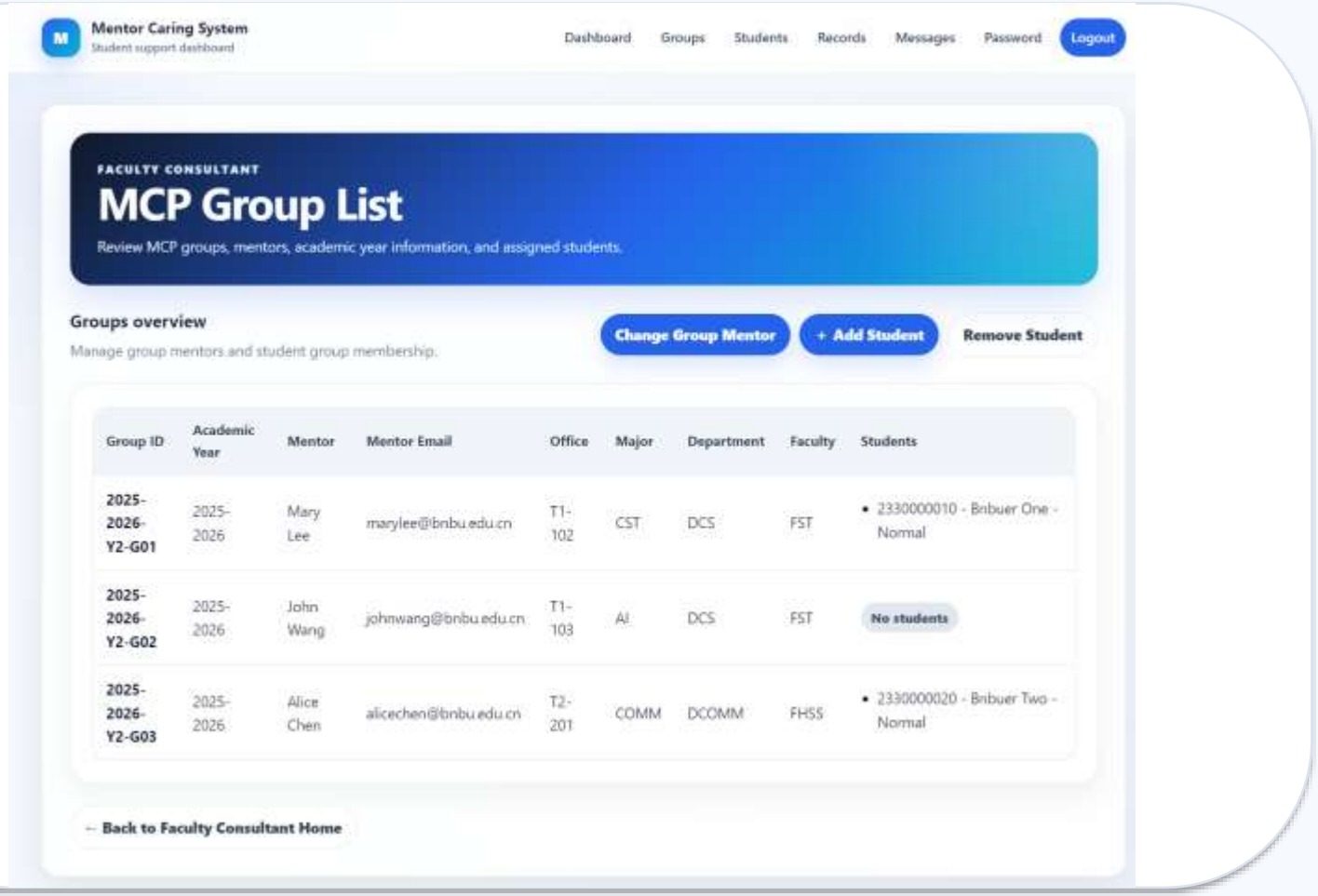
BENEFIT

Less repeated input

- Bulk import reduces manual typing.
- A preview table helps users check the data before activation.
- The system becomes the starting point for later records and appointments.

Feature 2 · Group Management

Maintain MCP groups and mentor ownership in one place



MCP Group List

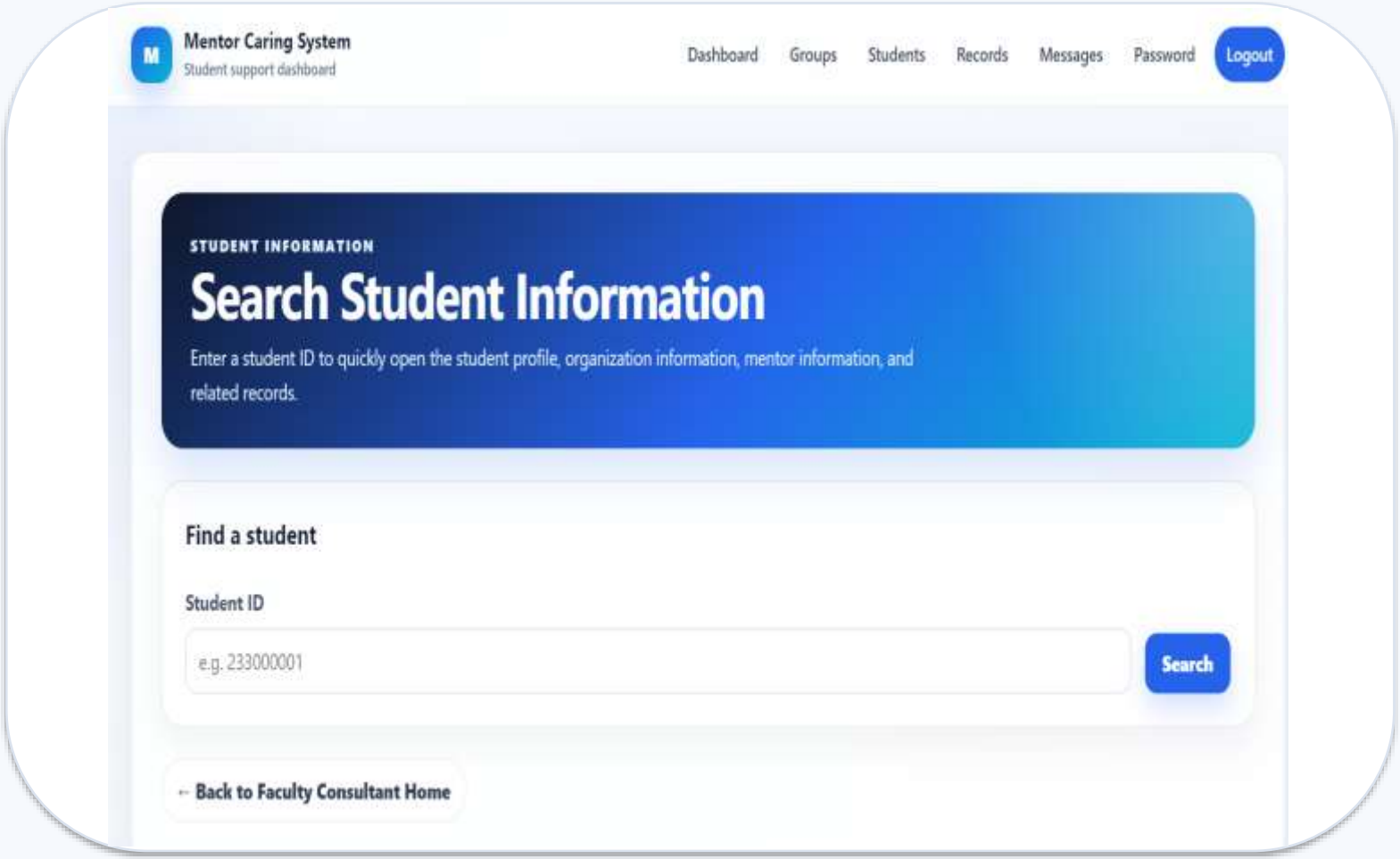
BENEFIT

Clear ownership

- Groups show academic year, faculty, mentor, and student count.
- Coordinators can update group members systematically.
- Mentoring responsibilities are easier to supervise.

Feature 3 · Fast Search

Search first, then act on student information



Search Student

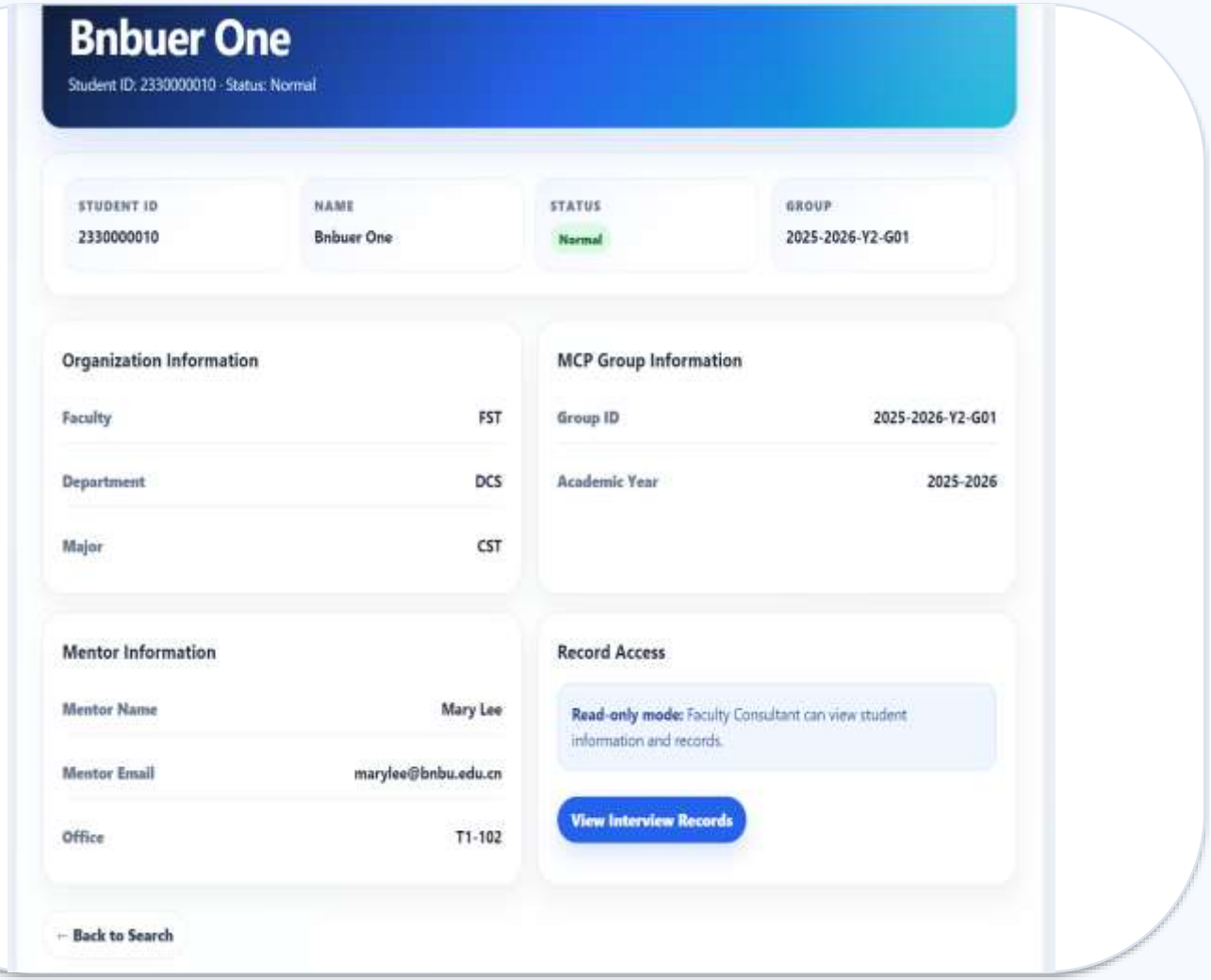
BENEFIT

Faster access

- Users can locate a student quickly by ID.
- The search page reduces time spent checking external files.
- Follow-up actions become easier after the right student is found.

Feature 4 · Student Profile

Connect profile information with mentoring records



Student Profile + Records

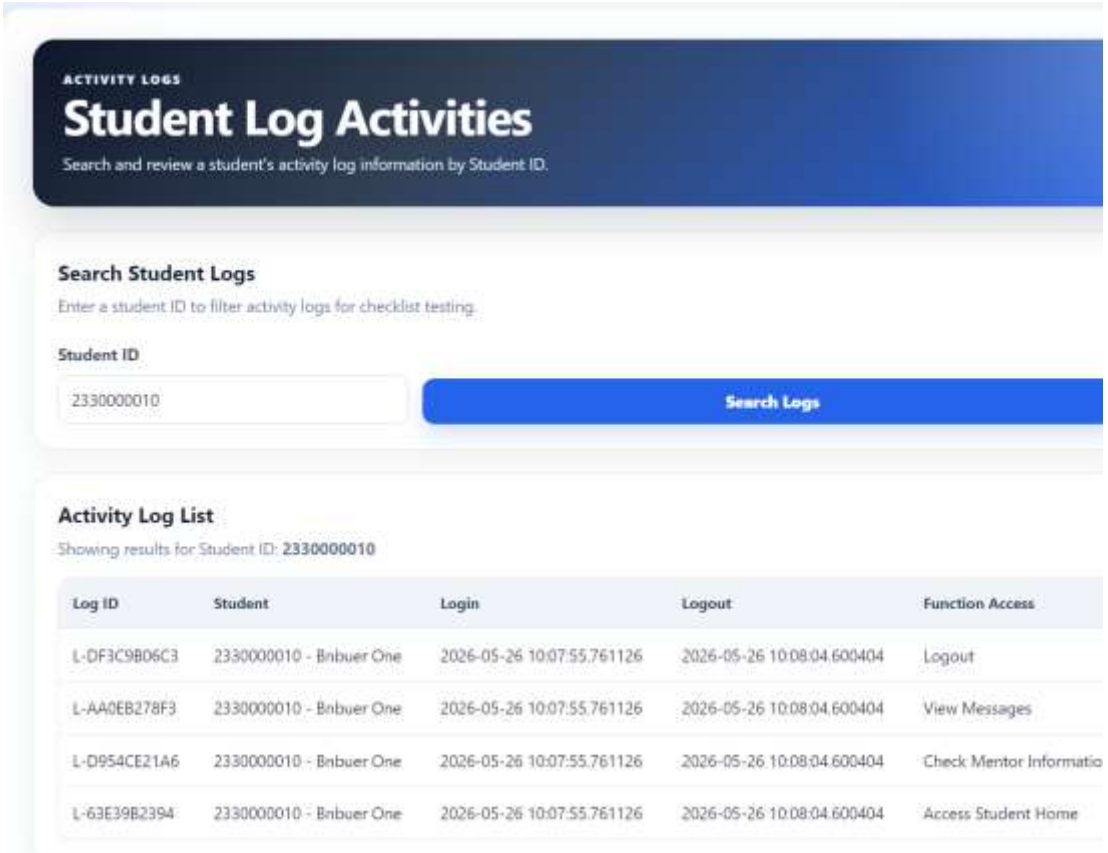
BENEFIT

Better decisions

- Student, organization, MCP group, and mentor information are visible together.
- Record access links support smoother follow-up.
- Mentors get context before taking action.

Feature 5 · Activity Logs

Make key user actions visible for supervision



Activity Logs

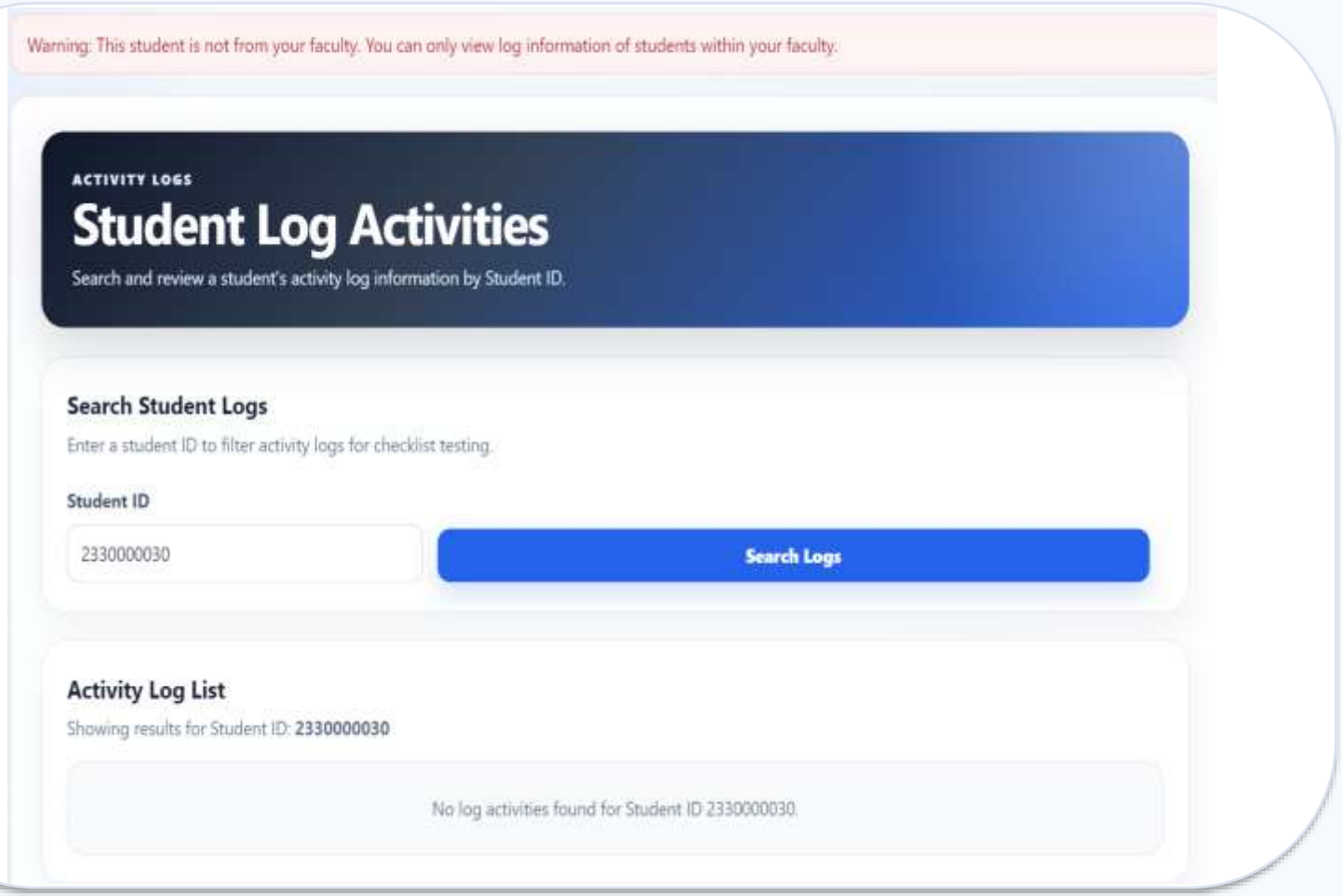
BENEFIT

Traceable workflow

- Logs show important actions and timestamps.
- Supervisors can check progress without asking every user manually.
- Accountability becomes part of the product experience.

Feature 6 · Permission Protection

Warn users when information is outside their scope



Permission Warning

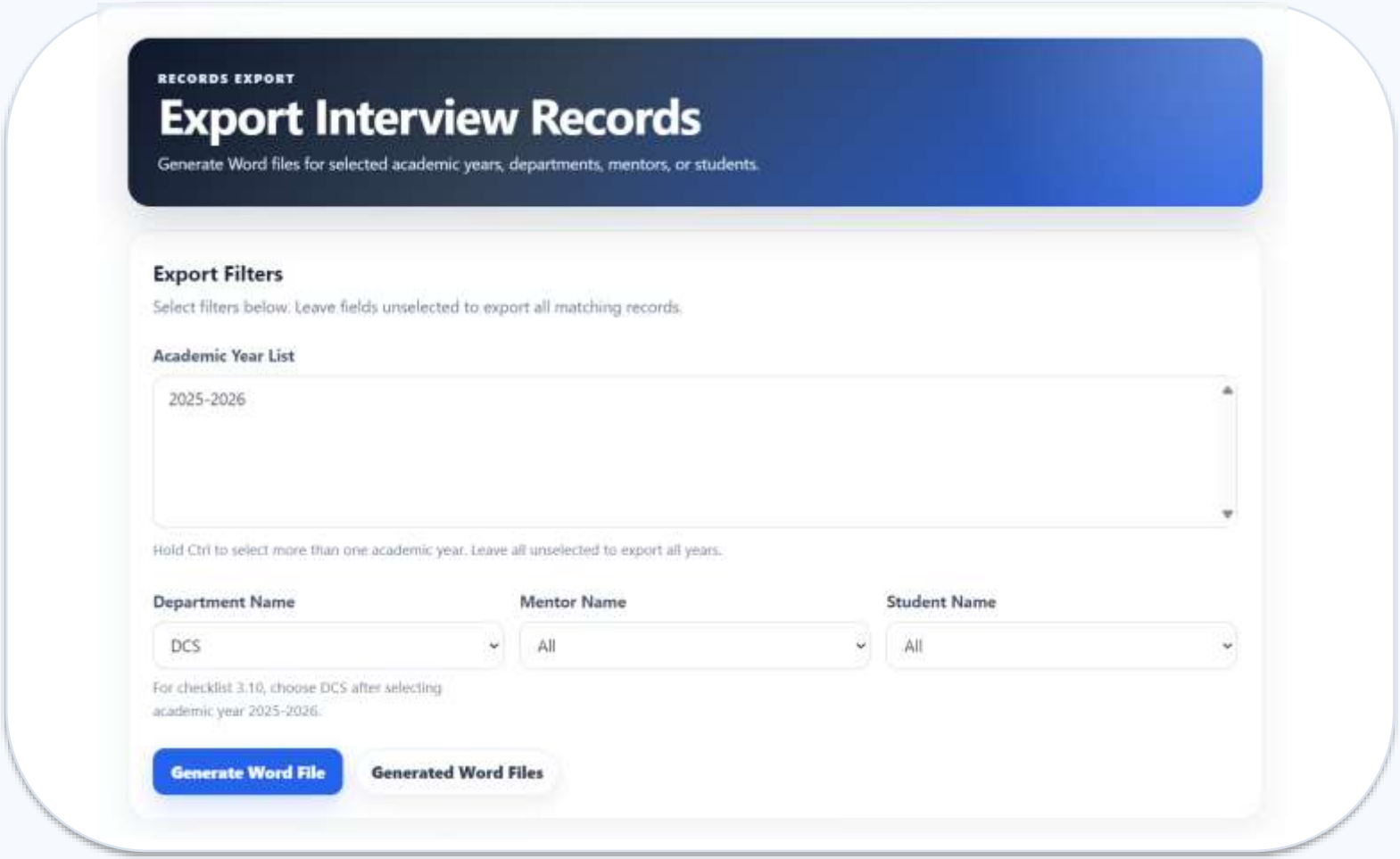
BENEFIT

Safer access

- The system signals access boundaries clearly.
- Users understand why some information cannot be viewed.
- This supports safer data handling in role-based work.

Feature 7 · Fast Reporting

Export selected interview records directly from system data



Word Export

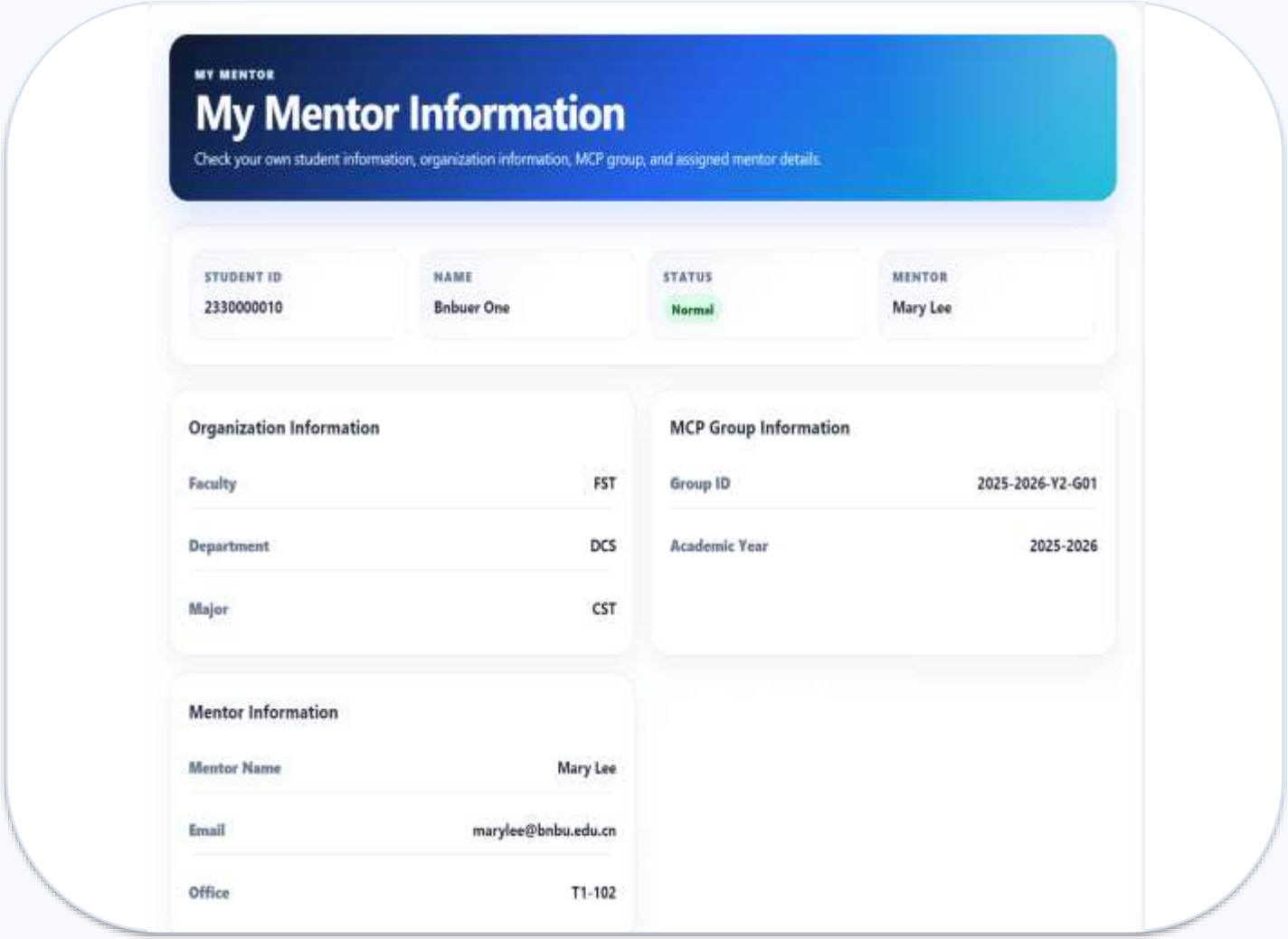
BENEFIT

Report-ready output

- Filters help select the right academic year, mentor, or student.
- Word generation reduces copying and formatting work.
- Faculty-level reports become faster and more consistent.

Student-Friendly Experience

Students can check mentor information inside the same platform



My Mentor Information

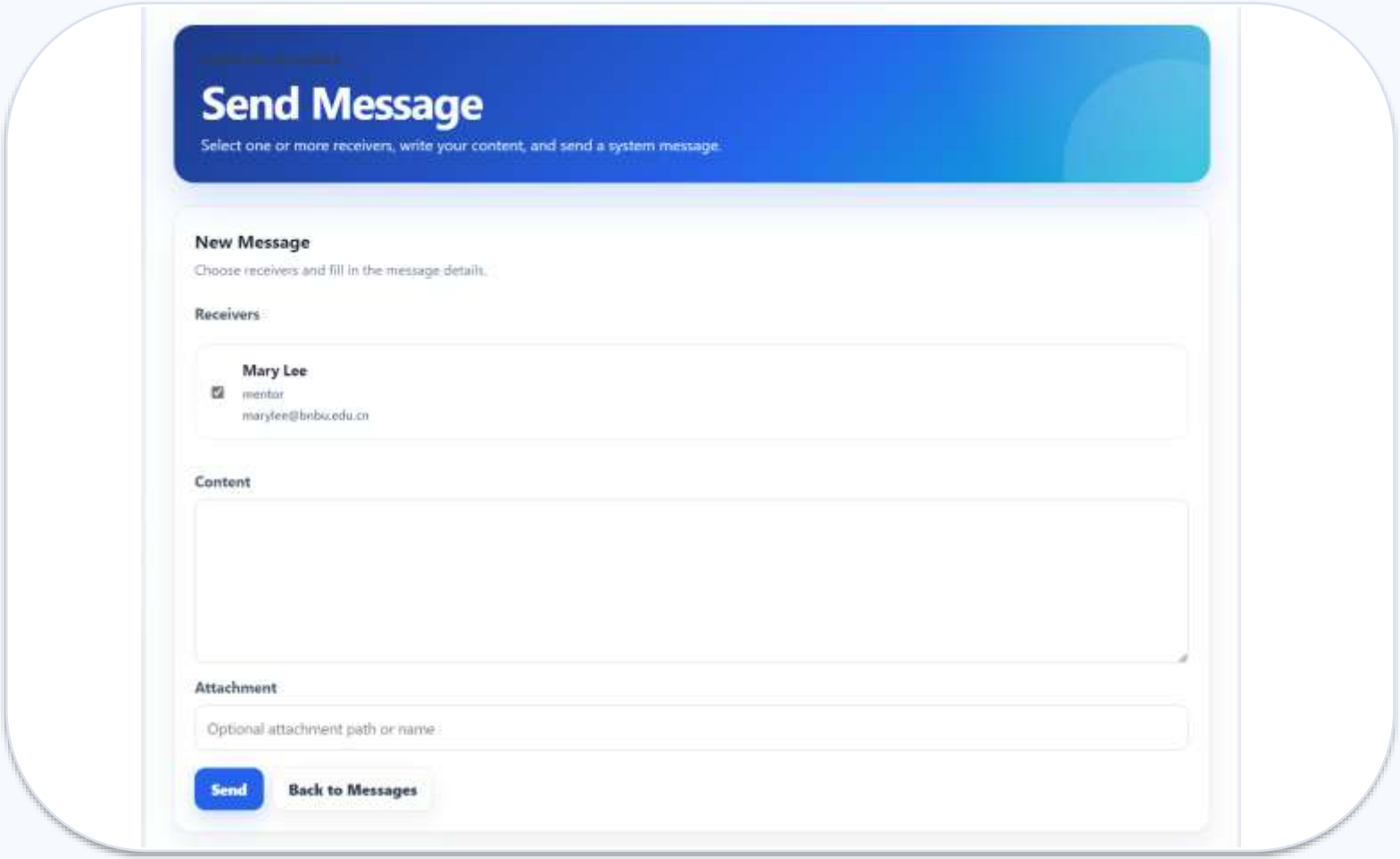
BENEFIT

No channel switching

- Students see mentor information and group details in one page.
- This reduces uncertainty about whom to contact.
- The product feels direct and student-centered.

Student-Friendly Communication

Send messages without leaving the system



Send Message

BENEFIT

Lower friction

- Receiver information is visible before sending.
- Message content and attachments are handled in one form.
- Communication becomes part of the mentoring workflow.

User-Friendliness Showcase

1

Mentor creates request

Select student, add content, provide available slots.

2

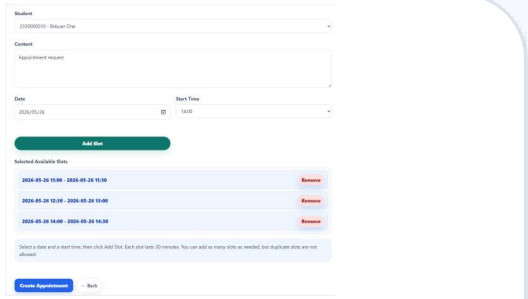
Student chooses time

Radio buttons make the selection direct and visible.

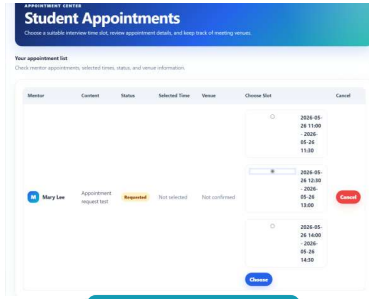
3

Mentor confirms venue

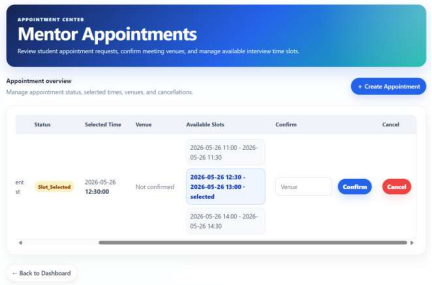
Selected time and venue are confirmed on the same page.



Mentor



Student

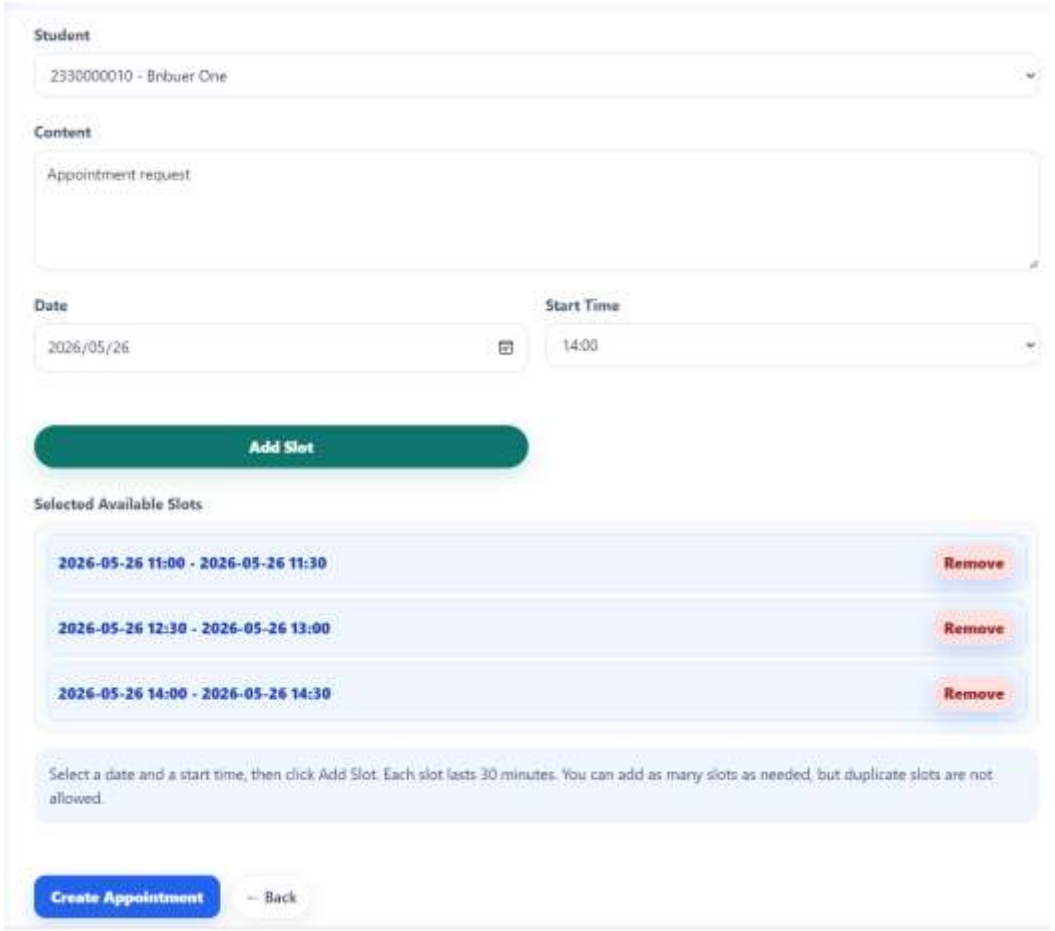


Confirm

User-friendliness is shown by the workflow: each role sees the exact action it needs next.

Showcase 1 · Mentor Creates Appointment

One clear form for one clear task



Create Appointment

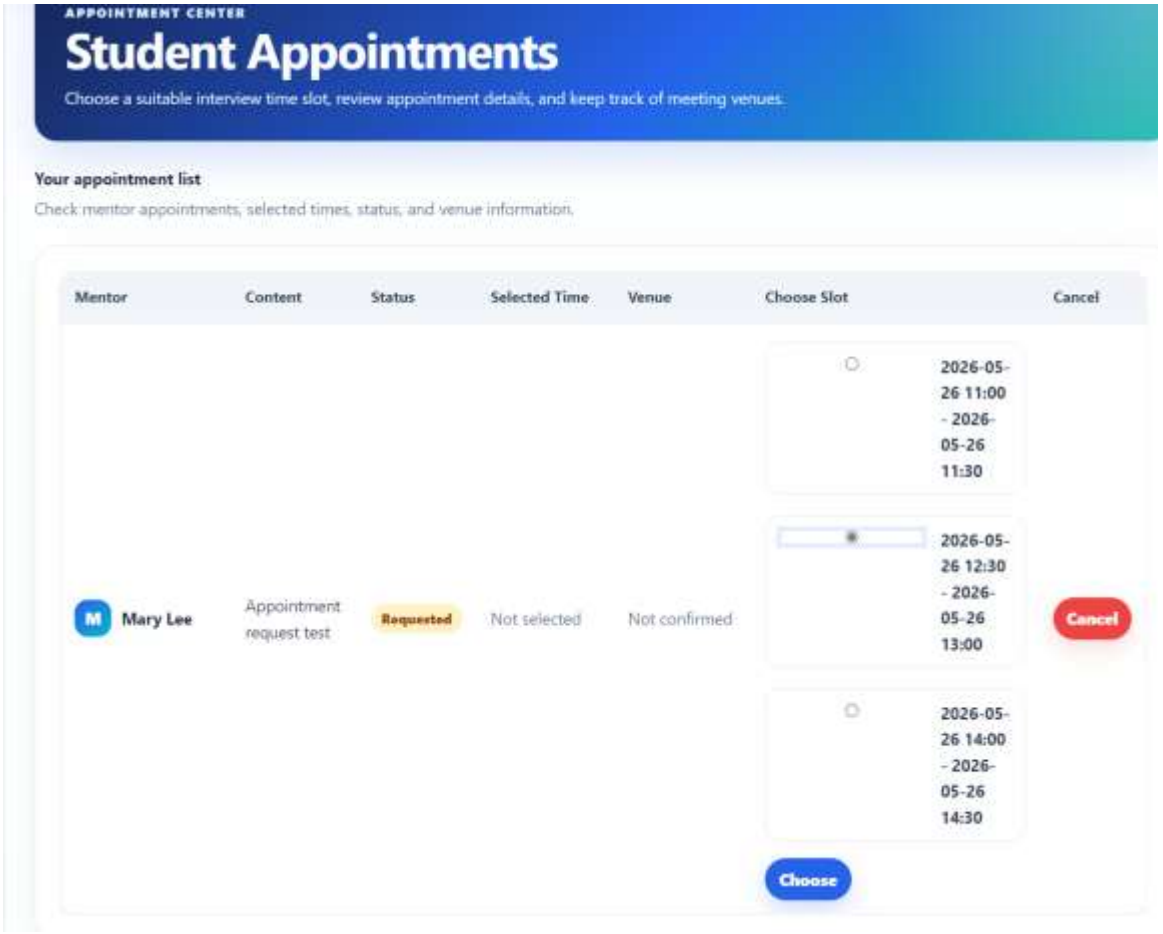
WHY IT WORKS

Fewer repeated messages

- The mentor selects a student and writes the appointment request.
- Several available slots can be added in the same form.
- Students receive structured choices instead of open-ended chat.

Showcase 2 · Student Chooses Time

The student can select a suitable slot directly



Student Appointments

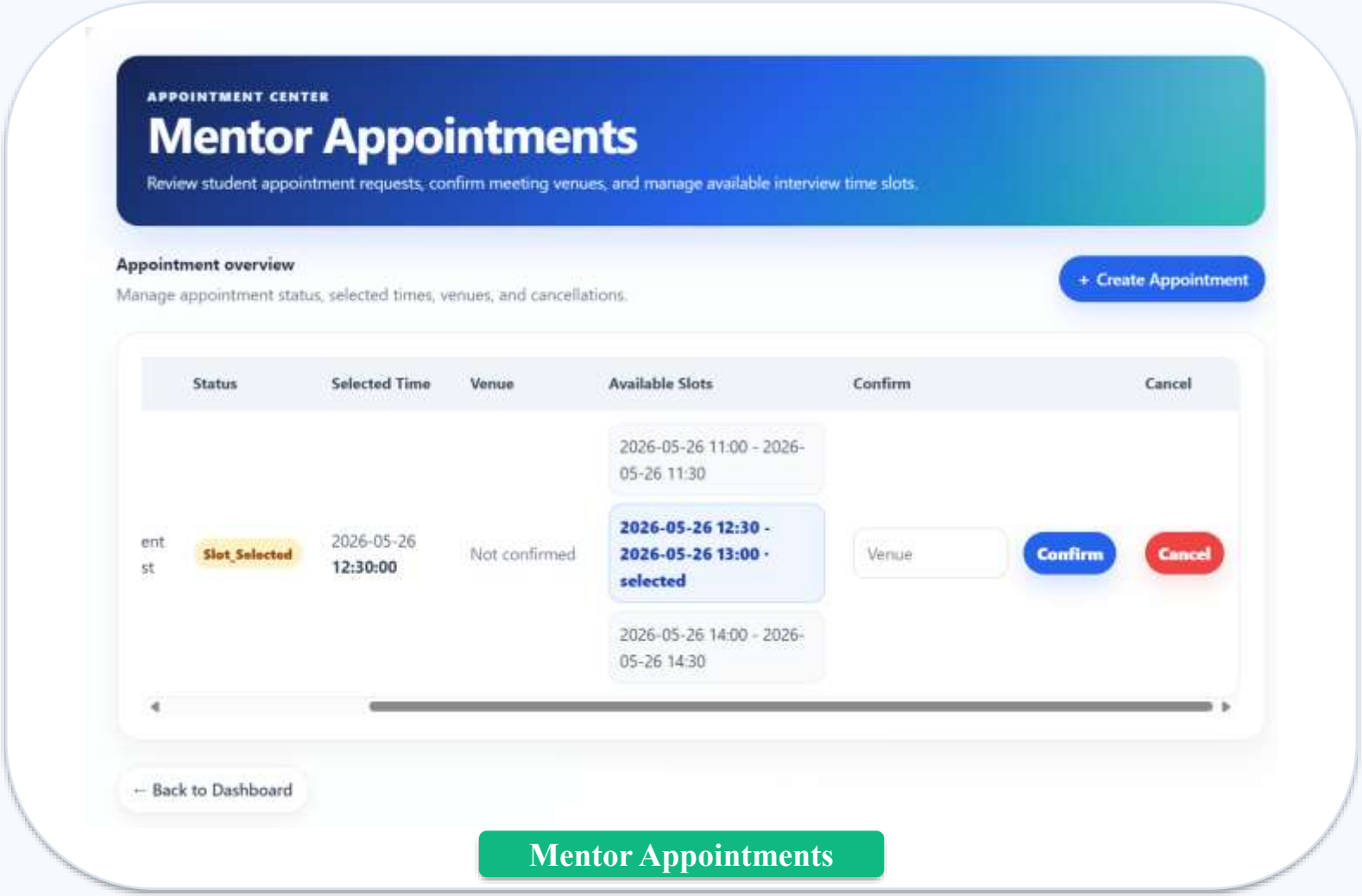
WHY IT WORKS

Easy decision

- Appointment status and available slots are shown together.
- Radio buttons make the choice simple.
- The student sees what to do next without asking the mentor again.

Showcase 3 · Mentor Confirms Venue

The mentor reviews the selected slot and finalizes the meeting



Mentor Appointments

BENEFIT

- Clearer appointment coordination
- Fewer confirmation mistakes
- Faster final decision

WHY IT WORKS

- Step-by-step confirmation flow
- Visible selected time and venue
- Simple role-specific actions

Code-Related Work

Mention only what supports the product and SE design.

Framework & tools

- Python Flask web app
- SQLAlchemy + SQLite
- Flask-Login
- Jinja templates + CSS
- Word export support

Code structure

- Role-based Flask Blueprints
- Templates + CSS for pages
- Models for users, records, messages, appointments
- Mappers for database operations
- Export module for Word reports

SE connection

- Implements MVC + Three-tier Architecture
- Boundary: pages and forms
- Control: workflow coordination
- Entity: core system data
- Mapper: database persistence

Part 3.3

ASDW: Live Demo

Show that the code behaves

Thanks